

Rapport de Projet de Fin de module
Application de collecte de déchets (ville / campus)



Réalisé par :

Sahmad Fatima Ezzahra

Encadré par :

Pr. Mohamed LACHGAR

Module : Projet de Développement Web et Mobile

Formation : Licence en Informatique

Semestre : S6

Année Universitaire : 2025–2026

Déclaration

Je soussignée, **Sahmad Fatima Ezzahra**, étudiante à l'École Normale Supérieure, Université Cadi Ayyad, Département d'Informatique, déclare que le présent rapport de projet de fin de modules, portant sur la conception et le développement d'une application de collecte de déchets, constitue le fruit de mon travail personnel.

L'ensemble des textes, figures, tableaux, extraits de code et illustrations présentés dans ce document ont été réalisés par mes soins, sauf indication contraire dûment référencée. Toutes les sources externes utilisées ont fait l'objet de citations et de références explicites. Je reconnais que tout manquement à cet engagement constituerait un acte de plagiat, considéré comme une infraction académique passible de sanctions conformément à la réglementation en vigueur.

J'autorise la mise à disposition d'un exemplaire de ce rapport à des fins pédagogiques, au bénéfice des étudiants et chercheurs futurs, et j'accepte qu'il soit consulté dans le cadre de l'intérêt général de l'enseignement supérieur et de la recherche.

Sahmad Fatima Ezzahra
Marrakech, le 26 avril 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à mon encadrant, Monsieur le Professeur Mohamed LACHGAR, pour son accompagnement bienveillant, ses conseils avisés et son soutien constant tout au long de la réalisation de ce projet. Sa disponibilité, sa rigueur et son expertise ont été des atouts précieux.

Je remercie chaleureusement ma mère pour son soutien indéfectible et ses encouragements constants.

Mes remerciements s'adressent également à l'ensemble des enseignants du département d'Informatique de l'École Normale Supérieure pour la qualité de leur formation.

Enfin, je remercie toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Abstract

This report presents the design and implementation of a waste collection management system for a city or campus environment. The project aims to improve the organization and efficiency of collection operations through a dedicated digital platform. The developed system, named CleanUp, is based on a modern architecture combining React for the web dashboard, React Native with Expo for the mobile application, PHP for the REST API, and MySQL for data storage. It integrates three user roles : administrator, supervisor, and driver. The platform was successfully tested and validated against all key functional requirements, including route management, real-time status updates, incident reporting, and performance indicator generation. API response times were measured under 250ms for standard requests, demonstrating its operational readiness.

Keywords : Waste collection, route management, mobile application, React Native, PHP, MySQL, dashboard, sustainable development

Résumé

Ce rapport présente la conception et la réalisation d'un système de gestion de la collecte des déchets pour un environnement urbain ou un campus universitaire. L'objectif est d'améliorer l'organisation et l'efficacité des opérations via une plateforme numérique dédiée. Le système développé, nommé CleanUp, repose sur une architecture moderne combinant React pour l'interface web, React Native avec Expo pour l'application mobile, PHP pour l'API REST, et MySQL pour le stockage. La plateforme intègre trois rôles : administrateur, responsable et chauffeur. Le système a été testé avec succès sur l'ensemble de ses fonctionnalités clés. Il permet la gestion des tournées, la mise à jour en temps réel des statuts, le signalement d'incidents et la consultation d'indicateurs de performance. Les temps de réponse de l'API sont inférieurs à 250 ms pour les requêtes standards, démontrant sa fiabilité opérationnelle.

Mots-clés : collecte des déchets, gestion des tournées, application mobile, React Native, PHP, MySQL, tableau de bord, développement durable

Liste des abréviations

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
Expo	Framework pour applications React Native
GPS	Global Positioning System
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
KPI	Key Performance Indicator
MVC	Model-View-Controller
MySQL	Système de gestion de base de données relationnelle
PDO	PHP Data Objects
PHP	Hypertext Preprocessor
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language

Table des figures

1.1	Diagramme de Gantt du projet CleanUp	9
2.1	Architecture trois couches du système d'information CleanUp	11
2.2	Interface de la base de données	13
2.3	Visualisation des points de collecte et des camions sur une carte	13
4.1	Architecture applicative du projet	22
4.2	Diagramme de classes	22
4.3	Diagramme de cas d'utilisation	23
5.1	Langages de programmation utilisés dans le projet	27
5.2	Architecture globale du système	29
5.3	Logo de l'application	30
5.4	Page de connexion	31
5.5	Page d'inscription	32
5.6	Dashboard principal	33
5.7	Gestion des points de collecte	33
5.8	Gestion des tournées (Routes)	34
5.9	Gestion des camions	35
5.10	Historique des collectes (Logs)	35
5.11	Gestion des utilisateurs — Interface Administrateur	36
5.12	Carte interactive des points de collecte — Vue Administrateur	37
5.13	Écran de connexion mobile	38
5.14	Tableau de bord agent	39
5.15	Ma tournée (My Route)	40

Liste des tableaux

1.1	Récapitulatif des objectifs du projet CleanUp	4
1.2	Organisation du projet en mode monôme	5
1.3	Planification détaillée du projet de collecte des déchets	7
2.1	Comparaison des solutions existantes et positionnement de CleanUp	14
3.1	Périmètre fonctionnel du système	17
3.2	Matrice des acteurs	19
4.1	Description des acteurs	24
5.1	Utilisation des fonctionnalités de React.js	27
5.2	Outils de développement utilisés	28
5.3	Stratégie de branches Git	28
5.4	Vue d'ensemble de l'architecture	29
5.5	Endpoints de l'API REST	30
5.6	Scénarios de tests fonctionnels	41
5.7	Variables d'environnement de configuration	42

Table des matières

Abstract	iii
Résumé	iv
Liste des abréviations	v
Table des figures	vi
Liste des tableaux	vii
Introduction Générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Présentation du projet	2
1.2.1 Sujet du projet	2
1.2.2 Intérêt du projet	2
1.3 Problématique et état de l'existant	3
1.3.1 Objectifs structurants	3
1.4 Objectifs du projet	3
1.4.1 Objectifs fonctionnels	4
1.4.2 Objectifs techniques	4
1.4.3 Objectifs opérationnels	4
1.5 Démarche de gestion de projet	5
1.5.1 Méthodologie	5
1.5.2 Planification du projet	5
1.5.3 Stratégie de réalisation	5
1.5.4 Planification du projet	6
1.5.5 Diagramme de Gantt	8
1.5.6 Conclusion	9
2 État de l'art	10
2.1 Introduction	10
2.2 Concepts fondamentaux	10
2.2.1 Systèmes d'information	10
2.2.2 Applications mobiles	11
2.2.3 Applications web	12
2.2.4 Systèmes de gestion de base de données	12
2.2.5 Géolocalisation	13
2.3 Solutions existantes	14
2.3.1 Analyse des principales solutions	14

2.4	Limites des solutions existantes	14
2.5	Positionnement du projet CleanUp	14
2.6	Conclusion	15
3	Étude Préliminaire et Fonctionnelle	16
3.1	Introduction	16
3.2	Contexte et périmètre du système	16
3.2.1	Contexte opérationnel	16
3.2.2	Périmètre fonctionnel	16
3.3	Étude des besoins par acteur	17
3.3.1	Administrateur	17
3.3.2	Responsable propreté	17
3.3.3	Agent / Chauffeur	17
3.4	Besoins fonctionnels	17
3.4.1	Liste des besoins	17
3.5	Scénarios d'utilisation	18
3.5.1	Scénario 1 : Connexion agent	18
3.5.2	Scénario 2 : Validation collective	18
3.5.3	Scénario 3 : Incident	18
3.6	Besoins fonctionnels	18
3.7	Matrice des acteurs	19
3.8	Conclusion	19
4	Analyse et conception	20
4.1	Introduction	20
4.2	Le formalisme UML	20
4.2.1	Frontend Web	21
4.2.2	Application Mobile	21
4.2.3	Backend	21
4.2.4	Base de données	21
4.3	Diagrammes de conception	21
4.3.1	Diagramme de classes	22
4.3.2	Diagrammes de cas d'utilisation	23
4.3.3	Cas d'utilisation Administrateur	24
4.3.4	Cas d'utilisation Responsable propreté	24
4.3.5	Cas d'utilisation Agent / Chauffeur	24
4.4	Descriptions textuelles des acteurs	24
4.5	Diagrammes de séquence	24
4.5.1	Gestion d'une tournée	25
4.5.2	Validation d'un point de collecte	25
4.5.3	Consultation du dashboard	25
4.6	Règles métier	25
4.7	Conclusion	25
5	Technologies et réalisation	26
5.1	Environnement de développement technique	26
5.1.1	Langages de programmation	26
5.1.2	Frameworks et bibliothèques	27
5.1.3	Base de données — MySQL	28
5.2	Environnement de développement logiciel	28
5.2.1	Outils principaux	28
5.2.2	Gestion de version avec Git	28

5.3	Architecture du système	29
5.3.1	Vue d'ensemble de l'architecture	29
5.3.2	Architecture de l'API REST	29
5.3.3	Flux de données — Scénario type	30
5.4	Implémentation et interfaces	30
5.4.1	Logo et identité visuelle	30
5.4.2	Interface web (React)	31
5.4.3	Application mobile (React Native)	38
5.5	Stratégie de tests	41
5.5.1	Tests d'intégration	41
5.5.2	Tests fonctionnels	41
5.5.3	Tests de performance	41
5.5.4	Tests de sécurité	42
5.6	Déploiement et configuration	42
5.6.1	Configuration locale (développement)	42
5.6.2	Instructions de déploiement	42
5.7	Conclusion	42
	Conclusion générale	44
	Bibliographie	47
	Appendices	50

Introduction Générale

La collecte des déchets constitue un service essentiel au bon fonctionnement de toute collectivité urbaine ou institutionnelle. Or, dans de nombreux contextes y compris celui des campus universitaires, cette activité repose encore sur des processus manuels peu efficaces : feuilles de route papier, communication par téléphone, absence de suivi en temps réel. Ce projet s'inscrit dans le champ des systèmes d'information opérationnels et des applications mobiles de terrain, en proposant une solution numérique intégrée pour la gestion et le suivi de la collecte des déchets.

La problématique centrale de ce projet est la suivante : comment améliorer la coordination, la traçabilité et l'efficacité des opérations de collecte des déchets grâce à une plateforme numérique adaptée aux besoins de chaque acteur (agent de terrain, responsable et administrateur) ? Les difficultés actuelles incluent l'absence de suivi en temps réel des tournées, la perte d'informations sur les incidents terrain et l'impossibilité de générer des indicateurs de performance fiables.

Pour répondre à ces enjeux, le projet vise à développer une application multi-plateforme (web React et mobile React Native) adossée à une API REST sécurisée (PHP / MySQL). L'objectif est de permettre la planification des tournées, le suivi GPS des camions, le signalement des incidents depuis le terrain et la visualisation des statistiques opérationnelles via un tableau de bord interactif.

La démarche progressive adoptée, organisée en neuf phases successives selon la planification établie, a permis de structurer le développement de manière itérative et cohérente, depuis l'analyse des besoins jusqu'à la validation finale du système.

Les résultats attendus sont une amélioration significative de l'efficacité opérationnelle, une traçabilité complète des opérations de collecte et une meilleure visibilité pour les responsables grâce aux indicateurs en temps réel. Le système doit également réduire les erreurs liées aux processus manuels et faciliter la prise de décision.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce premier chapitre présente le contexte général du projet de fin d'études intitulé « Application de collecte de déchets pour une ville ou un campus ». Il a pour objectif de proposer une solution numérique permettant d'améliorer l'organisation, le suivi et l'efficacité des opérations de collecte des déchets.

Dans de nombreux environnements urbains et universitaires, la gestion de la collecte des déchets souffre encore de plusieurs problèmes tels que le manque de coordination entre les équipes, l'absence de suivi en temps réel des tournées, ainsi que la difficulté de gestion des incidents sur le terrain.

Afin de répondre à ces limitations, ce projet propose une plateforme intelligente composée d'une application web (React) et mobile (React Native) permettant de digitaliser et optimiser l'ensemble du processus de collecte.

1.2 Présentation du projet

1.2.1 Sujet du projet

Le projet consiste à développer une plateforme numérique complète pour la gestion de la collecte des déchets. Elle combine une API REST sécurisée (PHP/JWT), une interface web dynamique (React), une application mobile cross-platform (React Native/Expo) et une base de données MySQL. L'application mobile permet aux agents de consulter leurs tournées, d'enregistrer l'état des points de collecte et d'envoyer leur position GPS. La plateforme web offre un tableau de bord avec des indicateurs de performance. Le projet vise une gestion centralisée, traçable et adaptée aux trois rôles : administrateur, responsable et agent.

1.2.2 Intérêt du projet

Ce projet présente plusieurs avantages significatifs pour les services de collecte des déchets :

- **Optimisation des opérations de collecte** : En proposant une planification numérique des tournées, une assignation des camions et un suivi en temps réel de l'avancement, le système réduit les omissions de collecte, élimine les doublons et améliore la réactivité face aux incidents terrain.
- **Suivi personnalisé par rôle** : Grâce à la gestion des rôles (*admin / responsable / agent*), chaque utilisateur dispose d'une interface adaptée à ses besoins opérationnels. L'agent accède à sa tournée depuis le mobile, le responsable consulte les indicateurs depuis le web, et l'administrateur gère l'ensemble du système ainsi que les accès.

- **Simplicité d'utilisation** : L'interface mobile React Native est conçue pour une prise en main rapide, accessible sur Android et iOS. La navigation par onglets (*Expo Router*) permet à l'agent de basculer facilement entre la liste des points, la carte GPS et le signalement d'incidents.
- **Gain de temps et d'efficacité** : En automatisant la création des logs de collecte, la mise à jour des statuts et la remontée des positions GPS, le système libère les agents des tâches administratives manuelles.
- **Amélioration continue** : Les données enregistrées dans *collection_logs* constituent une base stratégique permettant d'identifier les points problématiques récurrents, d'optimiser les routes et d'améliorer la qualité du service.
- **Confidentialité et sécurité des données** : Le projet implémente une authentification JWT sécurisée, un contrôle d'accès strict par rôle et une validation systématique des données entrantes.

En réunissant ces avantages, le projet CleanUp contribue à moderniser les services de propreté grâce à une solution numérique innovante, efficace et traçable.

1.3 Problématique et état de l'existant

À partir de l'analyse des processus existants de gestion des déchets et de leurs insuffisances, nous avons pu définir les objectifs structurants du projet CleanUp.

1.3.1 Objectifs structurants

Fonctionnels

- Suivi en temps réel des tournées de collecte avec mise à jour du statut des points (*collecté / non collecté / problème*).
- Gestion complète (CRUD) des points de collecte géolocalisés, des camions et des routes.
- Signalement des incidents terrain avec notes horodatées depuis l'application mobile.
- Gestion des utilisateurs avec contrôle d'accès par rôle.
- Tableau de bord opérationnel avec indicateurs de performance en temps réel.

Techniques

- Stack technologique : PHP REST API (backend), React (web), React Native / Expo (mobile), MySQL (base de données).
- Architecture MVC modulaire côté backend.
- Authentification stateless par JWT avec vérification des rôles.
- Sécurité des données : validation des entrées, protection CORS, mots de passe hashés.
- Performance : réponses API standardisées en JSON.

1.4 Objectifs du projet

Les objectifs offrent un cadre structurant pour l'ensemble du travail. Ils sont classés en trois catégories complémentaires.

1.4.1 Objectifs fonctionnels

- **OF1 — Gestion des points de collecte** : création, modification, suppression et consultation des points avec coordonnées GPS, zone, type de déchets et statut.
- **OF2 — Gestion des tournées et des camions** : création et planification des routes, association camion–route et suivi d’avancement.
- **OF3 — Suivi terrain via mobile** : consultation de la tournée, marquage des points (collecté / non collecté / problème), ajout de notes et envoi GPS.
- **OF4 — Journal de collecte automatique** : enregistrement de chaque passage dans `collection_logs` avec horodatage et position GPS.
- **OF5 — Géolocalisation** : affichage des positions des camions sur carte interactive (Leaflet.js + OpenStreetMap).
- **OF6 — Tableau de bord et reporting** : taux de collecte, incidents, camions actifs, historique filtrable.
- **OF7 — Gestion des utilisateurs** : création des comptes, attribution des rôles et désactivation des accès.

1.4.2 Objectifs techniques

- **OT1 — API REST sécurisée** : backend PHP avec point d’entrée unique, JWT et réponses JSON standardisées.
- **OT2 — Base de données MySQL** : schéma relationnel normalisé avec cinq tables principales — `users`, `collection_points`, `trucks`, `collection_routes`, `collection_logs`.
- **OT3 — Interface web React** : SPA avec composants réutilisables et communication API via Axios.
- **OT4 — Application mobile cross-platform** : React Native avec Expo Router, compatible Android et iOS.
- **OT5 — Architecture maintenable** : organisation MVC, validation centralisée et gestion des droits par rôle.

1.4.3 Objectifs opérationnels

- Réduire les omissions et doublons de collecte.
- Améliorer la réactivité face aux incidents terrain (délai moyen de signalement inférieur à 5 minutes).
- Permettre une prise de décision basée sur des KPI mesurables : taux d’exécution des tournées, nombre d’incidents par jour, taux de collecte par zone.
- Faciliter la coordination entre agents, responsables et administration.

Catégorie	Objectifs clés	Technologies impliquées
Fonctionnel	CRUD points, tournées, camions, suivi mobile, logs automatiques, GPS, dashboard	React, React Native, PHP API
Technique	API REST JWT, MySQL 4 tables, MVC, validation, CORS	PHP, MySQL, JWT, Expo Router
Opérationnel	Traçabilité, réactivité incidents, indicateurs KPI, coordination acteurs	React Dashboard, <code>collection_logs</code>

Table 1.1 : Récapitulatif des objectifs du projet CleanUp

1.5 Démarche de gestion de projet

1.5.1 Méthodologie

Ce projet a été réalisé de manière individuelle (monôme). Il repose sur une organisation personnelle structurée en phases progressives, permettant de maintenir une vision claire du projet tout en assurant une progression logique et maîtrisée du développement.

La démarche adoptée repose sur une organisation personnelle structurée, basée sur :

- planification des tâches
- progression par étapes
- la validation progressive des fonctionnalités

Cette approche a permis de :

- garder une vision claire du projet
- avancer de manière logique et organisée
- assurer une bonne maîtrise du développement

1.5.2 Planification du projet

Dans le cadre de ce projet de collecte des déchets, le travail a été réalisé de manière individuelle (monôme).

Ainsi, l'ensemble des responsabilités liées à la conception, au développement, aux tests et à la documentation a été assuré par une seule personne. Cette approche a nécessité une organisation rigoureuse, une bonne gestion du temps ainsi qu'une capacité à planifier efficacement les différentes phases du projet.

Le tableau suivant présente les rôles assurés :

Rôle	Personne en charge
Analyse des besoins	Sahmad Fatima Ezzahra
Conception du système	Sahmad Fatima Ezzahra
Développement (Frontend et Backend)	Sahmad Fatima Ezzahra
Gestion de la base de données	Sahmad Fatima Ezzahra
Tests et validation	Sahmad Fatima Ezzahra
Rédaction du rapport	Sahmad Fatima Ezzahra

Table 1.2 : Organisation du projet en mode monôme

1.5.3 Stratégie de réalisation

La réalisation du projet s'est appuyée sur une démarche progressive et itérative, organisée en plusieurs étapes successives. Chaque étape a été validée avant de passer à la suivante, afin de garantir la cohérence globale du système.

Cette stratégie repose sur :

- une décomposition du projet en tâches élémentaires
- une priorisation des fonctionnalités essentielles
- une validation continue des résultats obtenus

Cette approche a permis de limiter les erreurs, d'optimiser le temps de développement et d'assurer une meilleure qualité du produit final.

1.5.4 Planification du projet

Afin d'assurer le bon déroulement du projet, une planification détaillée a été mise en place. Le projet a été structuré en plusieurs phases principales couvrant l'ensemble du cycle de développement.

Phase	Objectifs et tâches
Phase 1 : Étude et analyse	▪ Analyse du problème de gestion des déchets ▪ Identification des besoins des utilisateurs (administrateur...) ▪ Définition des fonctionnalités principales (signalement, suivi, gestion) ▪ Étude des solutions existantes
Phase 2 : Conception	▪ Élaboration des diagrammes UML (cas d'utilisation, classes, séquences) ▪ Conception des maquettes des interfaces utilisateur ▪ Définition de l'architecture technique du système
Phase 3 : Développement Frontend	▪ Création des interfaces utilisateur (pages web) ▪ Intégration des maquettes en HTML, CSS et JavaScript ▪ Mise en place d'une navigation fluide et ergonomique
Phase 4 : Développement Backend	▪ Implémentation de la logique serveur en PHP ▪ Développement des fonctionnalités d'inscription et d'authentification ▪ Gestion des demandes de collecte
Phase 5 : Base de données	▪ Conception de la base de données (MySQL) ▪ Création des tables (utilisateurs, routes, camions) ▪ Gestion des relations entre les données
Phase 6 : Fonctionnalités avancées	▪ Suivi des demandes de collecte ▪ Interface d'administration (gestion des données) ▪ Amélioration de l'expérience utilisateur
Phase 7 : Tests et validation	▪ Tests fonctionnels de l'application ▪ Détection et correction des anomalies ▪ Vérification de la cohérence du système
Phase 8 : Documentation	▪ Rédaction du rapport technique ▪ Élaboration du manuel utilisateur ▪ Préparation des supports de présentation
Phase 9 : Finalisation	▪ Vérification globale du système ▪ Optimisation des performances ▪ Préparation de la maintenance

Table 1.3 : Planification détaillée du projet de collecte des déchets

1.5.5 Diagramme de Gantt

La planification du projet a été renforcée par l'utilisation d'un diagramme de Gantt, considéré comme l'un des outils les plus importants en gestion de projet. Cet outil permet de représenter de manière visuelle l'ensemble des tâches à réaliser, leur durée estimée ainsi que leur positionnement dans le temps. Grâce à cette représentation graphique, il devient plus simple d'organiser les activités, de suivre l'avancement du travail et de respecter les délais fixés.

Dans le cadre du projet CleanUp, le diagramme de Gantt a été utilisé dès les premières phases afin de structurer méthodiquement le déroulement du projet. Il a permis de découper le travail en plusieurs étapes successives et cohérentes, allant de l'analyse initiale jusqu'aux tests finaux et à la rédaction du rapport.

Ce diagramme permet principalement de représenter :

- les différentes phases du projet ;
- la durée prévisionnelle de chaque tâche ;
- les dates de début et de fin ;
- l'enchaînement chronologique des activités ;
- les dépendances entre certaines tâches ;
- l'état d'avancement global du projet.

Le recours au diagramme de Gantt a offert plusieurs avantages significatifs :

- une meilleure visibilité sur l'ensemble du projet ;
- une répartition claire des tâches dans le temps ;
- une anticipation des retards éventuels ;
- une organisation progressive et logique du travail ;
- une gestion plus efficace des priorités ;
- un meilleur suivi de l'avancement réel par rapport au planning initial.

Dans le cadre d'un travail réalisé en monôme, cet outil s'est révélé particulièrement utile. En effet, l'absence d'équipe impose une autonomie totale dans la gestion du temps, la priorisation des tâches et la résolution des problèmes rencontrés. Le diagramme de Gantt a ainsi servi de feuille de route tout au long du projet.

Il a également permis d'identifier les périodes critiques nécessitant davantage d'efforts, notamment durant les phases de développement technique, d'intégration entre le frontend et le backend, ainsi que durant la période de finalisation du mémoire.

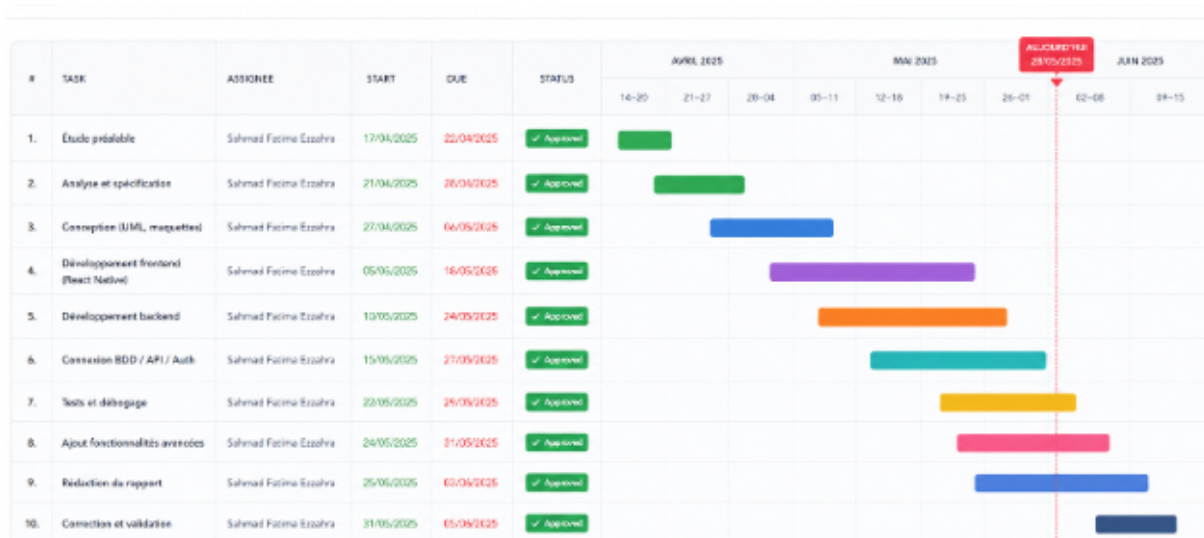


Figure 1.1 : Diagramme de Gantt du projet CleanUp

La figure 1.1 ci-dessous illustre cette planification. Les phases critiques identifiées sont le développement de l'API (phase 4), l'application mobile (phase 6) et la période d'intégration (fin phase 6 / début phase 7).

1.5.6 Conclusion

Ce chapitre a permis de présenter le contexte organisationnel général du projet ainsi que la méthodologie adoptée pour assurer sa réussite. Il a mis en évidence les différentes conditions de réalisation, les objectifs visés ainsi que les choix organisationnels effectués dès le démarrage du travail.

Le choix d'un développement en monôme a nécessité une gestion rigoureuse, une forte autonomie et une capacité permanente d'adaptation face aux difficultés rencontrées. L'ensemble des responsabilités — analyse, conception, développement, tests et rédaction — a été assumé de manière structurée selon une progression méthodique.

Par ailleurs, l'utilisation d'outils de gestion de projet, notamment le diagramme de Gantt, a permis d'améliorer considérablement l'organisation du travail. Cet outil a facilité la planification des tâches, le suivi de l'avancement et la maîtrise des délais tout au long du projet.

Ce chapitre a également montré l'importance d'une démarche méthodologique claire dans la conduite d'un projet informatique. En effet, la réussite d'un système ne dépend pas uniquement des compétences techniques, mais également d'une bonne organisation, d'une planification réaliste et d'un suivi continu.

Les éléments présentés dans ce chapitre constituent donc une base essentielle pour la suite du mémoire. Le chapitre suivant sera consacré à l'étude détaillée des besoins du système, à l'identification des acteurs principaux ainsi qu'à la conception fonctionnelle de l'application CleanUp.

Chapitre 2

État de l'art

2.1 Introduction

Dans un contexte marqué par l'urbanisation rapide, la croissance démographique et l'augmentation continue de la production des déchets ménagers et industriels, les collectivités territoriales sont confrontées à de nouveaux défis liés à la propreté urbaine [15] et à la préservation de l'environnement. La gestion efficace des déchets est devenue une priorité stratégique, car elle influence directement la qualité de vie des citoyens, l'image des villes ainsi que la protection des ressources naturelles.

Les méthodes traditionnelles de collecte reposent encore, dans de nombreux cas, sur des processus manuels ou partiellement numérisés. Les tournées sont souvent organisées de manière statique, les communications entre les équipes restent limitées, et les responsables disposent de peu d'outils pour superviser les opérations en temps réel. Cette situation engendre des pertes de temps, une mauvaise exploitation des ressources humaines et matérielles, ainsi qu'un manque de traçabilité des interventions.

Face à ces constats, la transformation numérique des services de propreté apparaît comme une solution pertinente. Grâce aux technologies web, mobiles, aux bases de données relationnelles et aux systèmes de géolocalisation, il devient possible d'optimiser les tournées, de suivre les opérations en direct et d'améliorer la prise de décision.

Le présent chapitre a pour objectif de présenter les concepts fondamentaux et les technologies utilisés dans le cadre du projet CleanUp. Il propose également une analyse des solutions existantes afin de mieux situer notre application dans son environnement technologique.

2.2 Concepts fondamentaux

2.2.1 Systèmes d'information

Un système d'information (SI) est un ensemble structuré de ressources humaines, matérielles, logicielles et organisationnelles permettant de collecter, stocker, traiter et diffuser l'information au sein d'une organisation.

Dans le domaine de la collecte des déchets, le système d'information permet de coordonner les différents acteurs :

- les agents de terrain ;
- les chauffeurs ;
- les responsables opérationnels ;
- les administrateurs ;
- les décideurs.

Il permet notamment :

- la centralisation des données relatives aux points de collecte ;
- le suivi en temps réel des opérations ;
- l'amélioration de la prise de décision ;
- la traçabilité des interventions ;
- la génération de rapports statistiques ;
- l'optimisation des ressources disponibles.

Dans le cadre du projet CleanUp, une architecture en trois couches a été adoptée :

- couche présentation (application web et mobile) ;
- couche logique métier (API REST) ;
- couche données (base de données MySQL).

Cette séparation garantit la modularité, la maintenabilité et l'évolutivité du système.

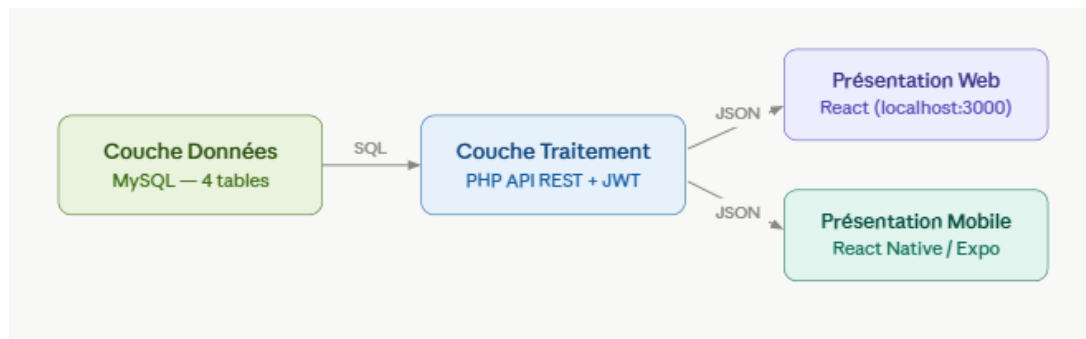


Figure 2.1 : Architecture trois couches du système d'information CleanUp

2.2.2 Applications mobiles

Les applications mobiles occupent aujourd'hui une place essentielle dans les systèmes nécessitant une interaction directe avec les agents sur le terrain. Elles permettent un accès instantané aux informations depuis un smartphone ou une tablette.

Dans le projet CleanUp, l'application mobile est développée avec React Native et Expo, ce qui permet de cibler Android et iOS avec une seule base de code.

Elle permet aux agents :

- de consulter les tournées assignées ;
- d'enregistrer l'état de chaque point de collecte ;
- de signaler les incidents rencontrés ;
- d'ajouter des notes terrain ;
- d'envoyer leur position GPS en temps réel ;
- de consulter leur historique d'activité.

Les avantages de cette approche sont :

- réduction des délais de transmission ;
- amélioration de la qualité des données ;
- diminution des erreurs humaines ;
- meilleure coordination entre équipes ;
- gain de temps opérationnel.

2.2.3 Applications web

L'application web constitue l'outil principal de gestion et de supervision destiné aux responsables et administrateurs.

Développée avec React, elle adopte le modèle SPA (*Single Page Application*) permettant une navigation fluide, rapide et moderne.

Ses principales fonctionnalités sont :

- gestion des points de collecte ;
- planification des tournées ;
- affectation des camions ;
- suivi en temps réel des opérations ;
- gestion des incidents ;
- consultation des statistiques ;
- exportation des rapports.

Cette interface transforme les données collectées sur le terrain en informations exploitables pour la prise de décision.

2.2.4 Systèmes de gestion de base de données

La base de données représente le noyau central du système d'information. Elle permet de stocker durablement les données nécessaires au fonctionnement de l'application.

Le projet CleanUp utilise MySQL, reconnu pour sa robustesse, sa performance et sa compatibilité avec PHP.

Les principales tables sont :

- users
- collection_points
- trucks
- collection_routes
- collection_logs

Ce modèle relationnel garantit :

- l'intégrité des données ;
- la cohérence des relations ;
- la traçabilité des opérations ;
- la sécurité des accès ;
- la facilité des requêtes analytiques.

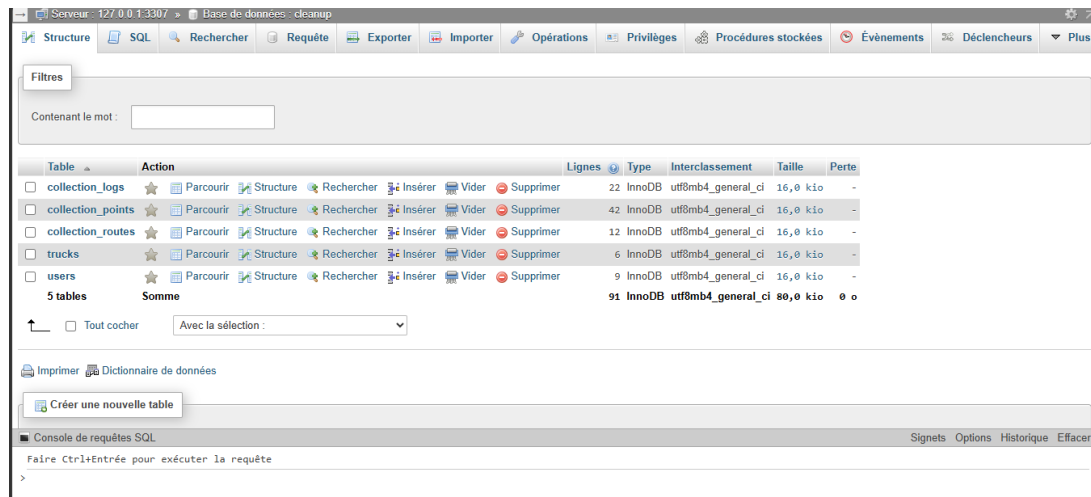


Figure 2.2 : Interface de la base de données

2.2.5 Géolocalisation

La géolocalisation repose sur l'utilisation du GPS afin de déterminer la position géographique des objets fixes ou mobiles.

Dans CleanUp, elle intervient à deux niveaux :

- géolocalisation statique des points de collecte ;
- géolocalisation dynamique des camions et agents.

Elle permet :

- une visualisation cartographique des opérations ;
- un suivi en temps réel ;
- l'optimisation des tournées ;
- la réduction des trajets inutiles ;
- une meilleure réactivité.

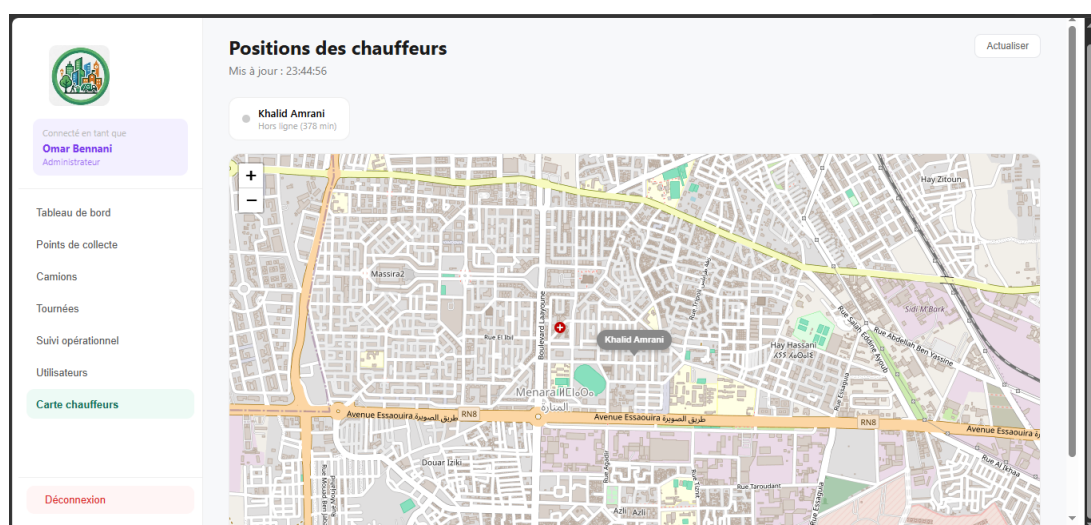


Figure 2.3 : Visualisation des points de collecte et des camions sur une carte

2.3 Solutions existantes

2.3.1 Analyse des principales solutions

Plusieurs solutions numériques de gestion des déchets sont disponibles sur le marché. Le tableau 2.1 présente une comparaison des principales d'entre elles en regard des besoins du projet CleanUp.

Table 2.1 : Comparaison des solutions existantes et positionnement de CleanUp

Solution	App mobile	Suivi GPS	Incidents	Open source	Coût
AMCS (?)	Oui	Oui	Partiel	Non	Élevé
Routemaster (?)	Oui	Oui	Non	Non	Moyen
Trackunit (?)	Oui	Oui	Non	Non	Élevé
Praxedo (?)	Oui	Partiel	Oui	Non	Moyen
CleanUp	Oui	Oui	Oui	Oui	Gratuit

2.4 Limites des solutions existantes

L'analyse des solutions actuelles met en évidence plusieurs limites :

- Manque d'intégration entre mobile, web et base de données ;
- Temps réel limité avec synchronisation tardive ;
- Interfaces complexes peu adaptées aux agents ;
- Gestion des incidents insuffisante ;
- Coûts élevés des licences ;
- Personnalisation réduite ;
- Dépendance fournisseur.

Ces limites justifient le développement d'une solution plus flexible et adaptée.

2.5 Positionnement du projet CleanUp

Le projet CleanUp se positionne comme une solution intégrée répondant aux besoins des collectivités locales, campus universitaires et structures intermédiaires.

Il propose :

- une application mobile intuitive ;
- une interface web complète ;
- un suivi en temps réel ;
- une base de données structurée ;
- un tableau de bord analytique ;
- une solution open source.

Ses principaux avantages sont :

- simplicité d'utilisation ;
- centralisation des données ;
- traçabilité complète ;
- réduction des coûts ;

- amélioration du service ;
- évolutivité.

Contrairement aux solutions existantes, CleanUp offre une intégration complète entre les différents composants du système.

2.6 Conclusion

Ce chapitre a permis de présenter de manière approfondie les concepts fondamentaux liés aux systèmes d'information, aux applications web et mobiles, aux bases de données relationnelles ainsi qu'aux technologies de géolocalisation utilisées dans le cadre du projet CleanUp.

L'étude des solutions existantes a mis en évidence plusieurs insuffisances majeures, notamment le manque d'intégration, la faiblesse du temps réel, la complexité d'utilisation et les coûts élevés de certaines plateformes.

Ces constats confirment la pertinence du développement d'une solution moderne, centralisée et adaptée aux besoins réels des acteurs de terrain. Le projet CleanUp répond à cette problématique en proposant un système intelligent combinant mobilité, supervision web et traçabilité complète des opérations.

Le chapitre suivant sera consacré à l'étude préliminaire et fonctionnelle du système, à travers l'identification des besoins, des acteurs et des principales exigences du projet.

Chapitre 3

Étude Préliminaire et Fonctionnelle

3.1 Introduction

L'étude préliminaire et fonctionnelle constitue l'une des phases les plus déterminantes du cycle de développement d'un système d'information. C'est au cours de cette étape que l'on passe d'une idée abstraite à une vision structurée et opérationnelle du projet.

Elle permet d'identifier avec précision les besoins des utilisateurs, de comprendre le domaine métier visé et de définir les fonctionnalités que devra offrir la future application.

Une étude fonctionnelle bâclée conduit souvent à des développements inadaptés et à un produit final ne répondant pas aux attentes. À l'inverse, une analyse rigoureuse permet de cadrer le projet et de guider efficacement les phases suivantes.

Dans ce projet d'application de collecte de déchets, l'objectif est d'optimiser la gestion des opérations de collecte à travers une solution numérique intégrée.

3.2 Contexte et périmètre du système

3.2.1 Contexte opérationnel

La gestion traditionnelle des déchets repose souvent sur :

- des plannings papier
- des communications téléphoniques
- des rapports manuels

Ces pratiques entraînent :

- une absence de traçabilité
- une mauvaise optimisation des tournées
- un manque de visibilité en temps réel

Le système proposé vise à digitaliser ces processus.

3.2.2 Périmètre fonctionnel

Le tableau ?? récapitule le périmètre fonctionnel du système en précisant pour chaque domaine les acteurs impliqués.

Domaine	Description	Acteurs
Gestion ressources	Utilisateurs, camions, points	Admin
Planification	Création des tournées	Responsable
Exécution	Collecte terrain	Agent
Supervision	Suivi en temps réel	Responsable
Reporting	Statistiques	Admin/Responsable
Incidents	Gestion problèmes	Responsable

Table 3.1 : Périmètre fonctionnel du système

3.3 Étude des besoins par acteur

3.3.1 Administrateur

L'administrateur assure la gestion globale du système.

- Gestion des utilisateurs (CRUD)
- Gestion des points de collecte
- Gestion des camions
- Consultation des statistiques
- Export des données

3.3.2 Responsable propreté

Le responsable planifie et supervise les opérations.

- Planification des tournées
- Affectation des camions
- Suivi en temps réel
- Gestion des incidents
- Consultation des KPI

3.3.3 Agent / Chauffeur

L'agent exécute les opérations via mobile.

- Consulter la tournée
- Marquer un point collecté
- Signaler un incident
- Envoyer la position GPS

3.4 Besoins fonctionnels

3.4.1 Liste des besoins

- BF-01 : Authentification sécurisée (JWT)
- BF-02 : Gestion des utilisateurs
- BF-03 : Gestion des points de collecte
- BF-04 : Gestion des camions
- BF-05 : Planification des tournées

- BF-06 : Suivi en temps réel
- BF-07 : Gestion des incidents
- BF-08 : Tableau de bord
- BF-09 : Historique des opérations
- BF-10 : Notifications

3.5 Scénarios d'utilisation

3.5.1 Scénario 1 : Connexion agent

- L'agent se connecte
- Consulte sa tournée
- Démarre la collecte

3.5.2 Scénario 2 : Validation collecte

- L'agent marque un point
- Le système enregistre GPS + heure

3.5.3 Scénario 3 : Incident

- Signalement d'un problème
- Notification envoyée au responsable

3.6 Besoins fonctionnels

Les besoins fonctionnels sont listés ci-dessous avec leurs préconditions et acteurs.

- BF-01 — Authentification JWT : *Précondition* : compte existant. *Acteurs* : tous. *Sortie* : token JWT valide (24 h), redirection vers le tableau de bord selon le rôle.
- BF-02 — Gestion des utilisateurs : *Précondition* : rôle administrateur. *Acteurs* : admin. *Opérations* : CRUD complet, attribution de rôle.
- BF-03 — Gestion des points de collecte : *Acteurs* : admin, responsable. CRUD avec coordonnées GPS, zone, type et statut.
- BF-04 — Gestion des camions : *Acteurs* : admin. CRUD avec immatriculation, modèle, état.
- BF-05 — Planification des tournées : *Acteurs* : responsable. Création de route, affectation camion, ajout de points.
- BF-06 — Suivi en temps réel : *Acteurs* : responsable. Carte interactive avec marqueurs colorés par statut.
- BF-07 — Gestion des incidents : *Acteurs* : agent (signalement), responsable (traitement). Note horodatée, entrée dans `collection_logs`.
- BF-08 — Tableau de bord : *Acteurs* : admin, responsable. KPI en temps réel, historique filtrable.
- BF-09 — Historique des opérations : consultation des logs par tournée, zone, date.
- BF-10 — Notifications : alertes au responsable lors d'un incident signalé (perspective d'évolution : push mobile).

3.7 Matrice des acteurs

Fonction	Admin	Responsable	Agent
Authentification	Oui	Oui	Oui
Gestion users	Oui	Non	Non
Points collecte	Oui	Oui	Oui
Tournées	Non	Oui	Oui
Suivi	Non	Oui	Oui
Incidents	Oui	Oui	Oui

Table 3.2 : Matrice des acteurs

3.8 Conclusion

Ce chapitre a permis de définir les besoins fonctionnels et non fonctionnels du système, d'identifier les trois acteurs principaux et leurs droits d'accès respectifs. Ces spécifications constituent la base des diagrammes UML présentés dans le chapitre suivant.

Chapitre 4

Analyse et conception

4.1 Introduction

La phase de conception constitue une étape essentielle dans le cycle de développement d'une application informatique. Elle permet de transformer les besoins exprimés lors de l'analyse fonctionnelle en une architecture claire, structurée et exploitable par les développeurs.

L'objectif principal de cette étape est de définir les différents composants du système, leurs interactions, la structure des données ainsi que les comportements attendus. Elle contribue également à réduire la complexité globale du projet et à garantir une meilleure qualité logicielle.

Dans ce chapitre, nous présentons les choix conceptuels adoptés pour la réalisation de notre projet Application de collecte de déchets, à travers plusieurs diagrammes UML.

4.2 Le formalisme UML

UML (*Unified Modeling Language*) est un langage standard de modélisation utilisé pour représenter graphiquement les systèmes informatiques.

Il permet de :

- visualiser la structure du système ;
- modéliser les interactions entre les acteurs ;
- documenter les décisions techniques ;
- faciliter la communication entre les membres de l'équipe ;
- préparer l'implémentation du projet.

UML n'est pas lié à un langage de programmation particulier, ce qui permet son utilisation avec différentes technologies.

Dans notre projet, UML a été utilisé pour représenter :

- les acteurs du système ;
- les cas d'utilisation ;
- les classes principales ;
- les séquences d'exécution ;
- l'architecture générale de l'application.

L'architecture de notre application repose sur une structure client/serveur séparant clairement les responsabilités entre le frontend, le backend et la base de données.

4.2.1 Frontend Web

La partie web est destinée au responsable propreté et à l'administrateur. Elle sera développée avec :

- React JS
- HTML5
- CSS3
- JavaScript

Elle permet la gestion des points de collecte, des tournées, du tableau de bord et du suivi opérationnel.

4.2.2 Application Mobile

Une application mobile cross-platform développée avec React Native et Expo est destinée aux agents / chauffeurs, compatible Android et iOS depuis une seule base de code JavaScript, afin de :

- consulter la tournée ;
- visualiser les points à collecter ;
- marquer un point collecté ou non collecté ;
- signaler un problème ;
- envoyer la position GPS.

4.2.3 Backend

Le backend assure la logique métier, le traitement des requêtes et la communication avec la base de données via une API REST.

4.2.4 Base de données

La base de données MySQL stocke toutes les informations du système.

Les principales tables sont :

- collection_points
- trucks
- collection_routes
- collection_logs

4.3 Diagrammes de conception

Cette section présente les principaux diagrammes UML réalisés pour modéliser le système.

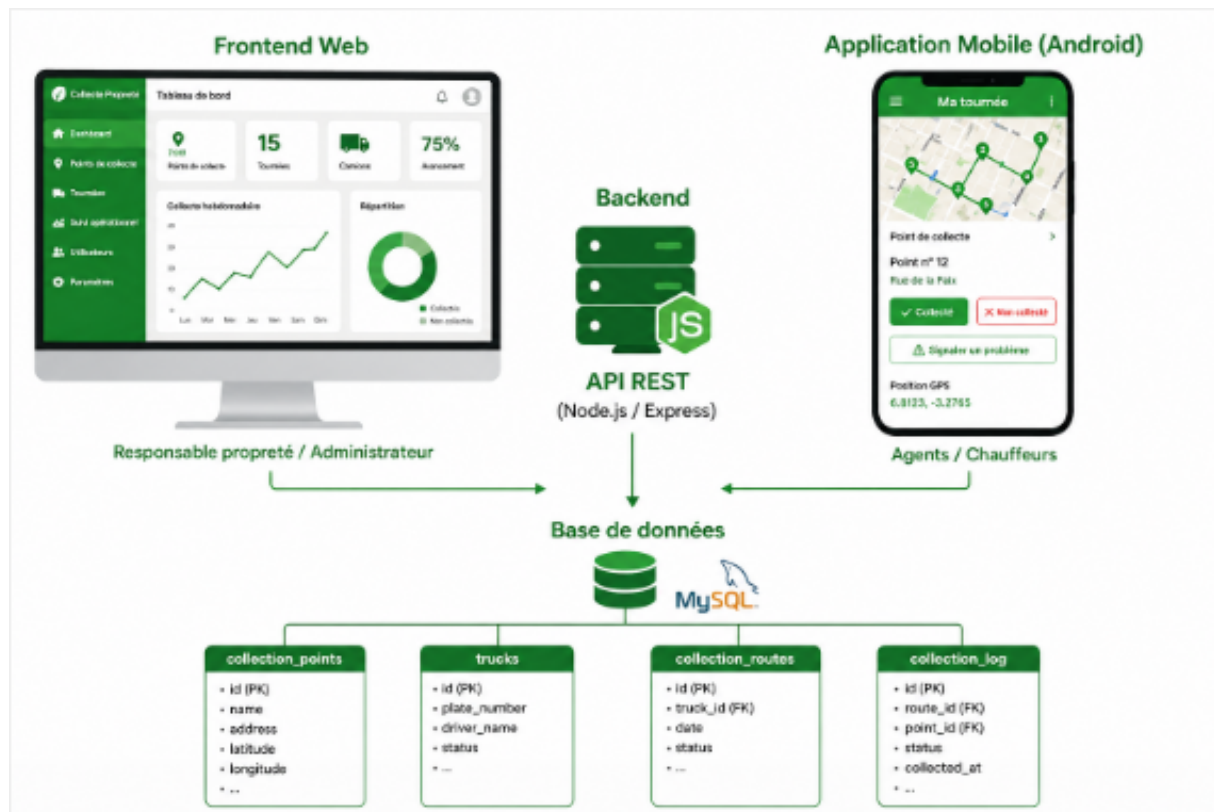


Figure 4.1 : Architecture applicative du projet

4.3.1 Diagramme de classes

Le diagramme de classes représente la structure statique du système. Il montre les entités principales, leurs attributs et les relations entre elles.

Les classes principales de notre système sont :

- CollectionPoint
- Truck
- CollectionRoute
- CollectionLog
- User

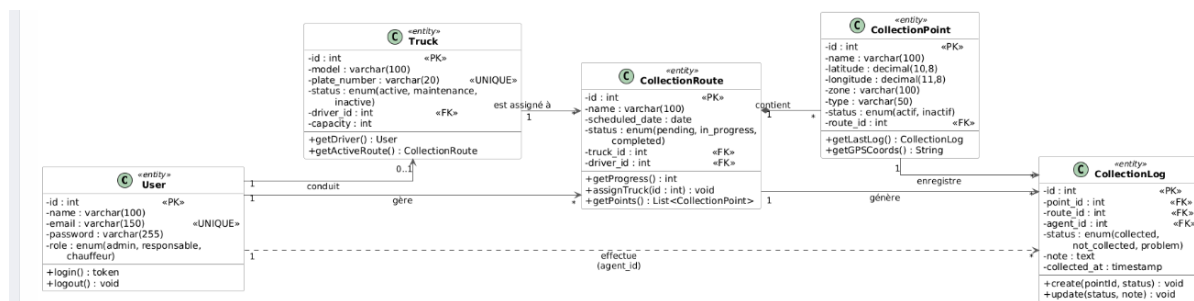


Figure 4.2 : Diagramme de classes

4.3.2 Diagrammes de cas d'utilisation

Ce chapitre a permis de définir les besoins fonctionnels et non fonctionnels du système, d'identifier les trois acteurs principaux et leurs droits d'accès respectifs. Ces spécifications constituent la base des diagrammes UML présentés dans le chapitre suivant.

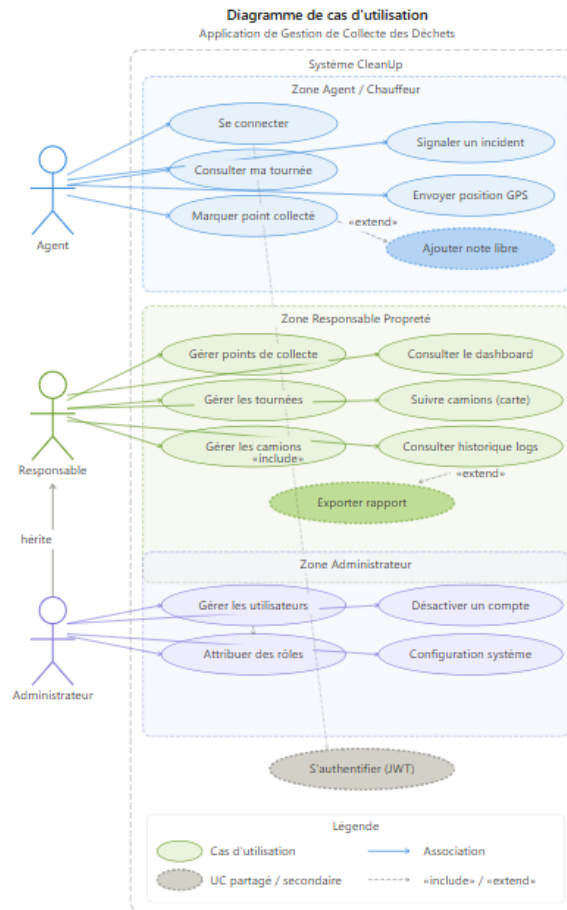


Figure 4.3 : Diagramme de cas d'utilisation

4.3.3 Cas d'utilisation Administrateur

L'administrateur peut :

- gérer les utilisateurs ;
- gérer les points de collecte ;
- gérer les camions ;
- consulter les statistiques ;
- superviser le système.

4.3.4 Cas d'utilisation Responsable propreté

Le responsable peut :

- créer les tournées ;
- affecter les camions ;
- suivre les opérations ;
- consulter les incidents ;
- consulter le tableau de bord.

4.3.5 Cas d'utilisation Agent / Chauffeur

L'agent peut :

- consulter sa tournée ;
- voir les points de collecte ;
- marquer un point collecté ;
- marquer non collecté ;
- signaler un problème ;
- ajouter une note ;
- envoyer la position GPS.

4.4 Descriptions textuelles des acteurs

Acteur	Description
Administrateur	Gère les utilisateurs, points de collecte, camions et supervise l'ensemble du système.
Responsable propreté	Planifie les tournées, suit l'état des collectes et consulte les statistiques.
Agent / Chauffeur	Exécute la tournée, collecte les déchets et met à jour les statuts sur mobile.

Table 4.1 : Description des acteurs

4.5 Diagrammes de séquence

Les diagrammes de séquence représentent l'ordre chronologique des interactions entre les acteurs et le système.

4.5.1 Gestion d'une tournée

Ce diagramme montre :

- création de la tournée ;
- affectation du camion ;
- consultation par l'agent ;
- début de collecte.

4.5.2 Validation d'un point de collecte

Ce diagramme représente :

- sélection du point ;
- choix du statut ;
- ajout éventuel d'une note ;
- enregistrement dans `collection_logs`.

4.5.3 Consultation du dashboard

Ce diagramme montre :

- récupération des statistiques ;
- calcul du taux de collecte ;
- affichage des camions actifs ;
- affichage des incidents.

4.6 Règles métier

Le système respecte les règles suivantes :

- Chaque point appartient à une zone ou une tournée.
- Chaque passage crée une entrée dans la table `collection_logs`.
- Un point déjà collecté ne peut pas être recollecté dans la même tournée sans justification.
- Un camion ne peut être affecté qu'à une seule tournée active.

4.7 Conclusion

Dans ce chapitre, nous avons présenté l'analyse et la conception de l'application de collecte de déchets à travers les différents diagrammes UML et les choix architecturaux retenus.

Cette phase nous a permis de structurer le système, définir les relations entre les composants et préparer efficacement la phase de réalisation.

Le chapitre suivant sera consacré aux outils, technologies utilisées ainsi qu'à l'implémentation du projet.

Chapitre 5

Technologies et réalisation

Ce chapitre présente les différentes technologies, outils et méthodes utilisés pour la réalisation de l'application de gestion de la collecte des déchets. Il décrit l'environnement technique du projet, les choix technologiques effectués, ainsi que l'architecture globale du système. Ensuite, il détaille l'implémentation des différentes composantes (backend, frontend web et application mobile), et se termine par la stratégie de tests adoptée afin de garantir la fiabilité du système.

5.1 Environnement de développement technique

5.1.1 Langages de programmation

PHP — Langage Backend

PHP (Hypertext Preprocessor) est le langage principal utilisé côté serveur. Il assure la gestion de la logique métier, le traitement des requêtes HTTP, et la communication avec la base de données MySQL via PDO. Sa compatibilité native avec Apache et MySQL, sa large communauté et sa maturité en font un choix stratégique pour des projets académiques et professionnels.

- Version utilisée : PHP 8.1+ avec support des types stricts et des attributs natifs.
- Points forts exploités dans ce projet :
 - Gestion des sessions et authentification JWT
 - Validation et filtrage des données entrantes
 - Construction d'une API REST complète
 - Gestion des erreurs HTTP avec codes de statut appropriés

JavaScript — Langage Frontend

JavaScript (ES2022+) est utilisé pour rendre les interfaces web et mobiles interactives. Combiné avec React et React Native, il permet de développer des applications réactives et performantes à partir d'une seule base de compétences.

- Gestion des états et des événements
- Communication asynchrone avec l'API via fetch et axios
- Manipulation du DOM et des composants React
- Gestion des erreurs et des promesses

CSS et Tailwind CSS — Styles et Design

CSS (Cascading Style Sheets) définit l'apparence visuelle des pages web. Dans ce projet, Tailwind CSS est utilisé comme framework utilitaire permettant un design rapide, responsive et cohérent. Tailwind propose des classes préfabriquées directement dans le JSX, réduisant considérablement le volume de CSS personnalisé à écrire.

- Grille responsive (flex, grid)
- Composants stylisés : boutons, cartes, badges, tableaux
- Thème de couleurs personnalisé (vert pour l'écologie)
- Support du mode sombre (dark mode)



Figure 5.1 : Langages de programmation utilisés dans le projet

5.1.2 Frameworks et bibliothèques

React.js — Interface Web

React est une bibliothèque JavaScript open source développée par Meta, utilisée pour construire des interfaces utilisateur sous forme de composants réutilisables. Son modèle de Virtual DOM garantit des mises à jour efficaces de l'interface sans rechargement complet de la page.

Fonctionnalité	Utilisation dans le projet
Composants fonctionnels	Tableaux, formulaires, modales, cartes
Hooks (useState, useEffect)	Gestion d'état et appels API
React Router	Navigation entre pages du dashboard
Context API	Gestion globale de l'utilisateur connecté
Axios	Requêtes HTTP vers le backend PHP

Table 5.1 : Utilisation des fonctionnalités de React.js

React Native (Expo) — Application Mobile

React Native permet de développer des applications mobiles natives (Android et iOS) à partir d'un code JavaScript partagé. L'utilisation d'Expo simplifie considérablement le démarrage du projet, la gestion des dépendances natives et le déploiement.

- Expo Go pour le développement et les tests rapides
- Navigation avec React Navigation (Stack, Tab)
- Accès aux API natives : GPS, caméra, notifications
- AsyncStorage pour la persistance locale des données
- Expo Location pour le suivi GPS en temps réel

Tailwind CSS

Tailwind CSS est utilisé pour le design rapide et responsive de l'interface. Il propose des classes préfabriquées directement dans le JSX, réduisant considérablement le volume de CSS personnalisé à écrire.

5.1.3 Base de données — MySQL

MySQL est le système de gestion de base de données relationnelle (SGBDR) utilisé pour stocker l'ensemble des données opérationnelles du système. Son architecture éprouvée, ses performances sur des volumes de données moyens, et son intégration native avec PHP en font le choix idéal pour ce projet. MySQL est utilisé pour stocker les données (utilisateurs, routes, camions, points de collecte, logs).

5.2 Environnement de développement logiciel

5.2.1 Outils principaux

Outil	Rôle	Version
Visual Studio Code	Éditeur de code principal	1.85+
XAMPP	Serveur local Apache + MySQL	8.2.4
GitHub / Git	Gestion de version et collaboration	2.43+
Postman	Test et documentation des API REST	10.x
Expo CLI	Démarrage et gestion de l'app mobile	6.x
phpMyAdmin	Interface graphique pour MySQL	5.x

Table 5.2 : Outils de développement utilisés

5.2.2 Gestion de version avec Git

Le projet suit une stratégie de branches Git structurée pour organiser le développement :

Branche	Rôle
main	Code stable, prêt pour la production
feature/web-dashboard	Développement du dashboard React
feature/mobile-agent	Développement de l'app mobile
feature/api-backend	Développement de l'API PHP

Table 5.3 : Stratégie de branches Git

5.3 Architecture du système

Le système repose sur une architecture client–serveur trois tiers, séparant clairement les responsabilités entre la couche présentation (frontend), la couche logique métier (backend API) et la couche données (base de données).

5.3.1 Vue d'ensemble de l'architecture

Couche	Composant	Technologie	Rôle
Présentation Web	Frontend Web	React.js + Tailwind	Interface responsable/admin
Présentation Mobile	App Mobile	React Native (Expo)	Interface agent terrain
Logique Métier	API REST	PHP 8.1	Traitement et règles métier
Données	Base de données	MySQL 8.0	Persistance des données
Serveur	Serveur Web	Apache (XAMPP)	Hébergement et routage

Table 5.4 : Vue d'ensemble de l'architecture

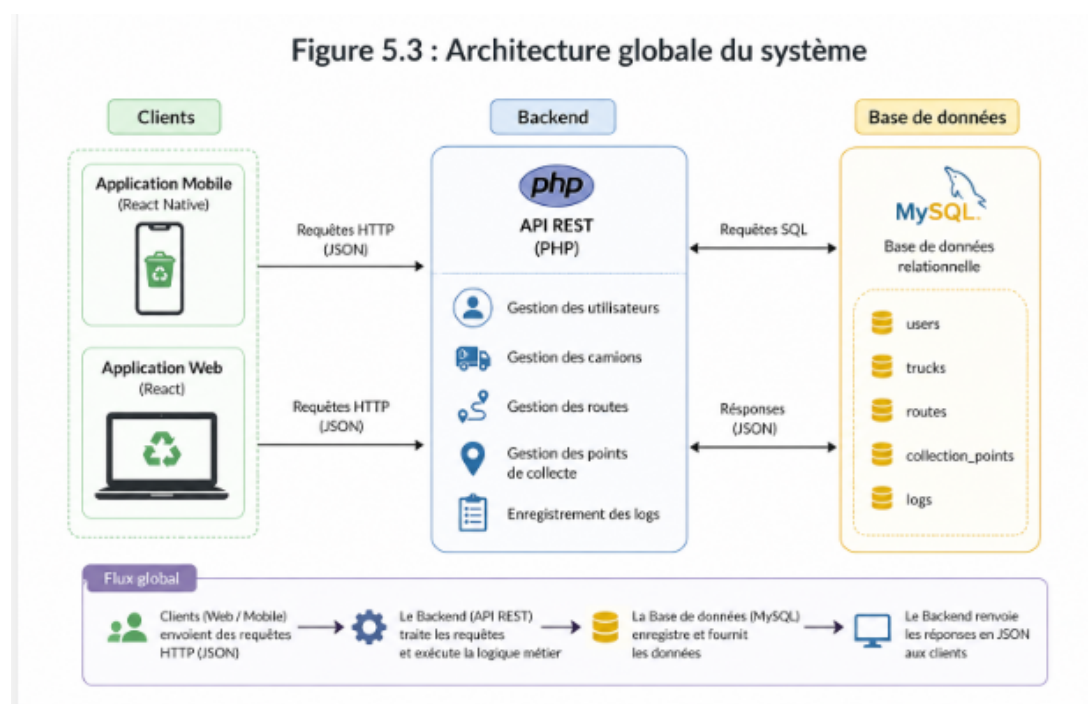


Figure 5.2 : Architecture globale du système

5.3.2 Architecture de l'API REST

L'API REST PHP constitue le cœur du système. Elle expose des endpoints sécurisés consommés par les deux clients (web et mobile). L'authentification est assurée par JWT (JSON Web Tokens).

Endpoint	Méthode	Description	Accès
/api/auth/login	POST	Authentification	Public
/api/points	GET/POST	Liste et création des points	Admin/Resp.
/api/points/{id}	PUT/DELETE	Modification/suppression	Admin/Resp.
/api/routes	GET/POST	Gestion des tournées	Admin/Resp.
/api/routes/{id}	POST	Assigner un camion	Admin
/api/trucks	GET/POST/PUT	Gestion des camions	Admin
/api/logs	GET/POST	Historique des collectes	Tous
/api/logs/route/{id}	GET	Logs d'une tournée	Agent/Resp.
/api/dashboard/stats	GET	Statistiques du dashboard	Admin/Resp.

Table 5.5 : Endpoints de l'API REST

5.3.3 Flux de données — Scénario type

Voici le flux de données complet lors d'une opération de collecte par un agent :

1. L'agent se connecte via l'application mobile (POST /api/auth/login) → JWT retourné
2. L'app mobile récupère la tournée du jour (GET /api/routes?agent=X) → liste des points
3. L'agent se déplace vers un point de collecte et met à jour le statut (POST /api/logs)
4. L'API crée une entrée dans collection_logs avec timestamp et position GPS
5. Le responsable visualise en temps réel l'avancement sur le dashboard web
6. En fin de tournée, le statut passe automatiquement à 'terminée'

5.4 Implémentation et interfaces

5.4.1 Logo et identité visuelle



Figure 5.3 : Logo de l'application

Le logo de notre application de gestion de la collecte des déchets a été conçu pour représenter les valeurs principales du système : propreté, organisation et durabilité environnementale.

L'élément visuel central du logo symbolise une poubelle stylisée entourée d'un cercle de recyclage, représentant le cycle complet de la gestion des déchets (collecte, transport et traitement). Les couleurs utilisées (vert et bleu) évoquent respectivement l'écologie et la technologie, reflétant ainsi la nature intelligente du système développé.

Ce logo permet d'identifier facilement la plateforme et renforce son identité visuelle en tant que solution moderne dédiée à l'optimisation des services de collecte urbaine.

5.4.2 Interface web (React)

Page de connexion

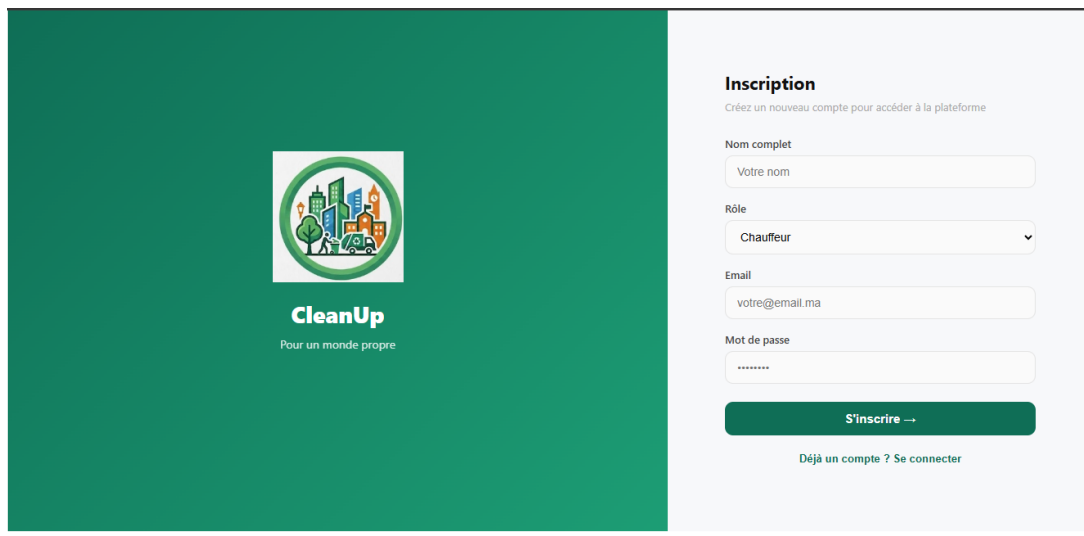


Figure 5.4 : Page de connexion

La page de connexion constitue le point d'entrée du système web. Elle permet aux utilisateurs (administrateur ou responsable) de s'authentifier à l'aide de leurs identifiants.

Cette interface garantit la sécurité d'accès grâce à l'utilisation d'un système d'authentification basé sur JWT. Une fois connectés, les utilisateurs sont redirigés vers le tableau de bord principal.

- Validation côté client (champs requis, format email)
- Feedback utilisateur : messages d'erreur, spinner de chargement
- Redirection automatique vers le dashboard après connexion
- Gestion de session : déconnexion automatique à expiration du JWT

Page d'inscription

La page d'inscription permet à l'administrateur de créer de nouveaux comptes utilisateurs pour accéder à la plateforme CleanUp. Elle est accessible depuis la page de connexion via le lien « *Déjà un compte ? Se connecter* ».

Cette interface collecte les informations nécessaires à la création d'un compte et garantit leur validité avant toute soumission au serveur. Le formulaire est structuré en quatre champs principaux :

- Nom complet : champ texte libre permettant de saisir l'identité de l'utilisateur (ex : « Votre nom »)

- **Rôle** : liste déroulante permettant de sélectionner le profil de l'utilisateur parmi les rôles disponibles dans le système (Chauffeur, Responsable, Administrateur). Ce champ détermine les droits d'accès attribués au compte créé.
- **Email** : adresse électronique servant d'identifiant de connexion (ex : « votre@email.ma »). Ce champ est soumis à une validation de format côté client.
- **Mot de passe** : champ sécurisé avec masquage des caractères saisis. Le mot de passe est hashé côté serveur avant d'être stocké en base de données.

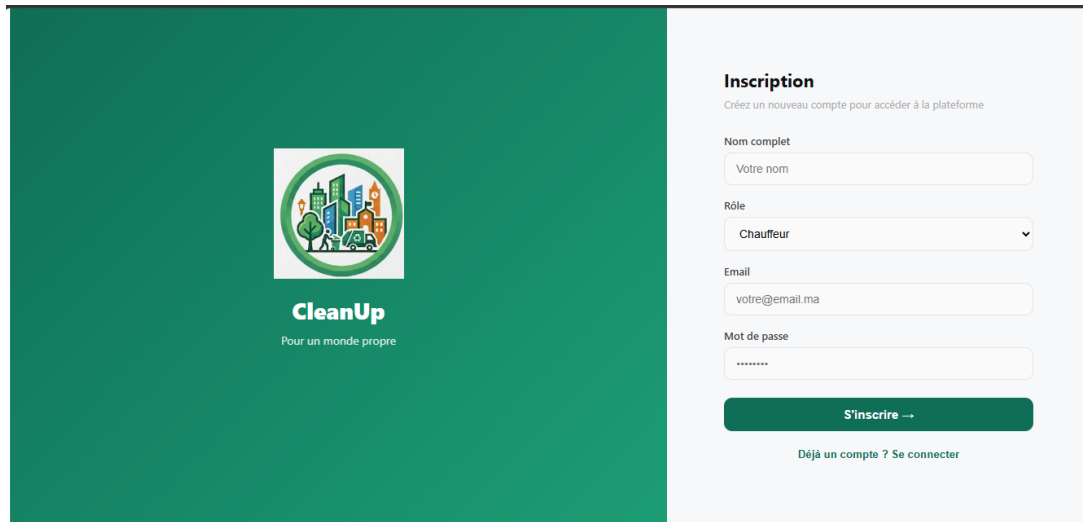


Figure 5.5 : Page d'inscription

La soumission du formulaire déclenche un appel à l'endpoint POST `/api/auth/register` de l'API REST. En cas de succès, l'utilisateur est redirigé vers la page de connexion. Les fonctionnalités de sécurité de cette page incluent :

- **Validation côté client** : vérification des champs requis, format email et longueur minimale du mot de passe
- **Validation côté serveur** : vérification de l'unicité de l'email dans la base de données via `Validator.php`
- **Hashage du mot de passe** avant stockage en base de données MySQL
- **Contrôle d'accès** : la création de comptes avec le rôle *Administrateur* est réservée aux administrateurs authentifiés
- **Feedback utilisateur** : messages d'erreur explicites en cas d'email déjà utilisé ou de données invalides

L'interface suit la même charte graphique que la page de connexion : panneau gauche vert avec le logo CleanUp et la devise « *Pour un monde propre* », panneau droit blanc avec le formulaire. Cette cohérence visuelle renforce l'identité de la plateforme.

Dashboards principal

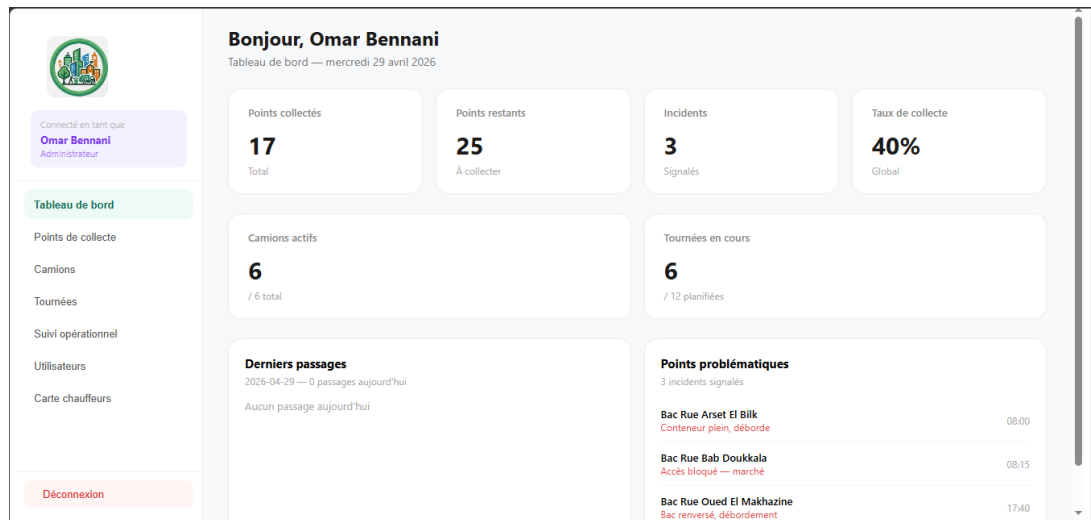


Figure 5.6 : Dashboard principal

Le dashboard représente l'interface centrale du système web. Il permet de visualiser en temps réel les indicateurs clés du système, notamment :

- Nombre de camions actifs
- Nombre de points de collecte
- État des tournées
- Statistiques globales de collecte

Cette interface offre une vue globale permettant de suivre efficacement l'activité du système.

Gestion des points de collecte (CRUD)

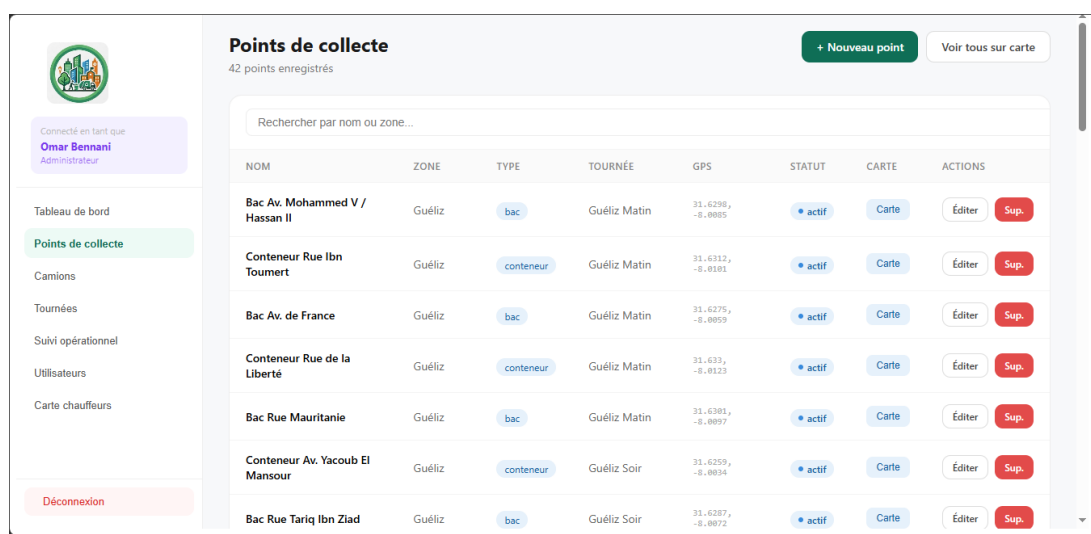


Figure 5.7 : Gestion des points de collecte

Cette interface permet la gestion complète des points de collecte (CRUD) :

- Ajouter un nouveau point avec coordonnées GPS
- Modifier les informations d'un point
- Supprimer un point
- Consulter la liste des points par zone ou type

Chaque point est associé à une tournée afin d'organiser la collecte de manière structurée.

Gestion des tournées (Routes)

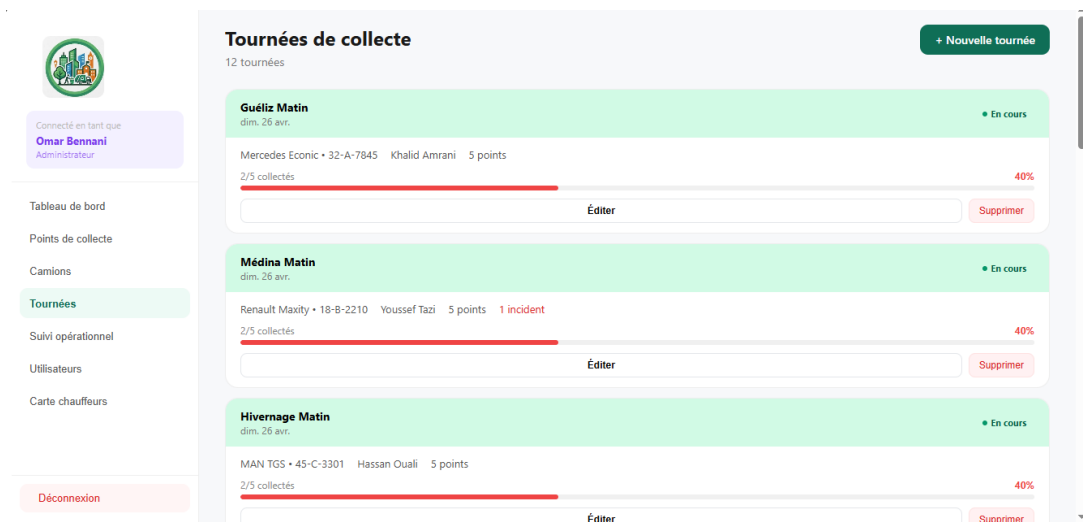


Figure 5.8 : Gestion des tournées (Routes)

Cette page permet de créer et gérer les tournées de collecte :

- Créer une nouvelle route
- Associer un camion à une tournée
- Affecter des points de collecte
- Suivre l'état de la tournée (en attente, en cours, terminée)

Cette interface est essentielle pour l'organisation des opérations sur le terrain.

Gestion des camions

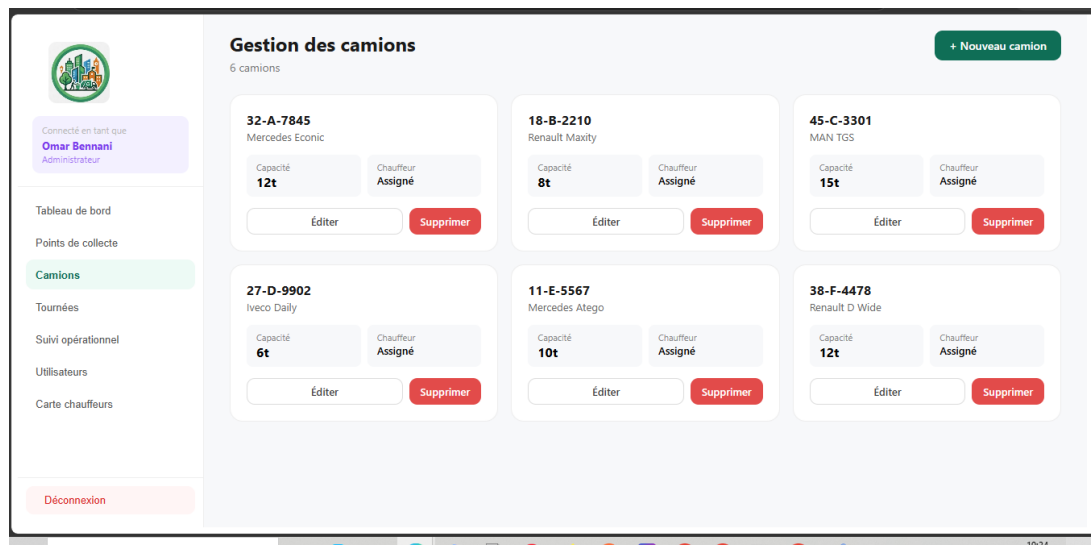


Figure 5.9 : Gestion des camions

Cette interface permet la gestion de la flotte des véhicules :

- Ajout de camions avec immatriculation et modèle
- Modification des informations du véhicule
- Affectation d'un chauffeur au camion
- Suivi de l'état du véhicule (actif, maintenance, inactif)

Historique des collectes (Logs)

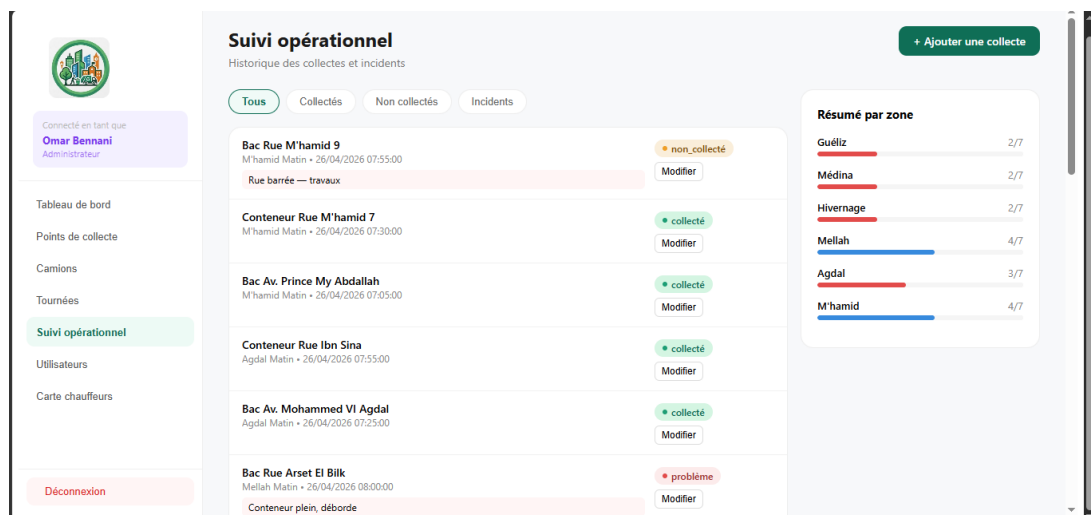


Figure 5.10 : Historique des collectes (Logs)

Cette interface affiche l'historique complet des opérations de collecte. Elle permet de consulter :

- Les points déjà collectés
- Les incidents signalés

- Les notes des agents
- Les dates et heures de passage

Elle joue un rôle important dans le suivi et l'analyse des performances.

Gestion des utilisateurs (Admin)

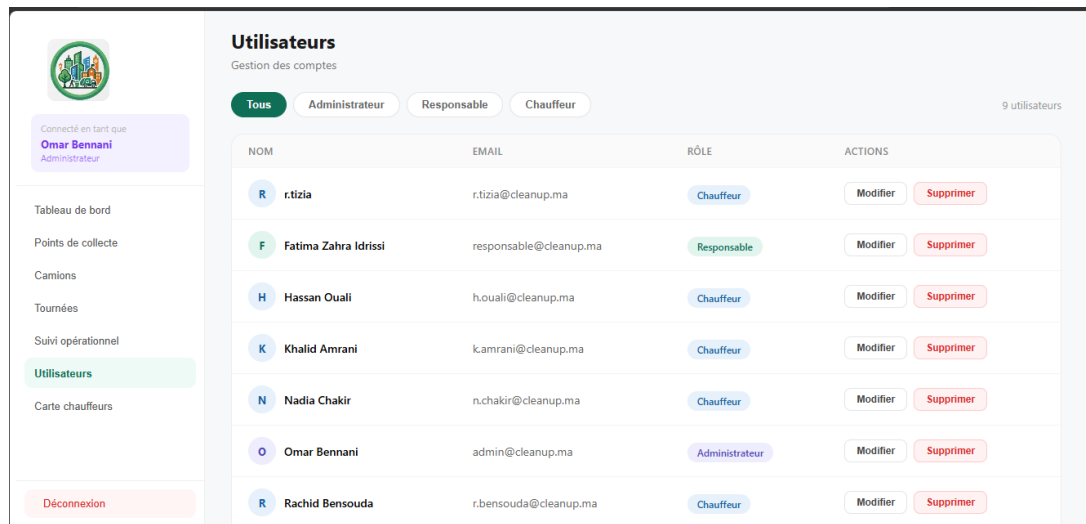


Figure 5.11 : Gestion des utilisateurs — Interface Administrateur

L'interface de gestion des utilisateurs est réservée exclusivement à l'administrateur du système. Elle constitue le point central de contrôle des accès et des droits au sein de l'application CleanUp. Cette interface offre les fonctionnalités suivantes :

- Consulter la liste des utilisateurs : affichage de tous les comptes enregistrés avec leur nom, email et rôle attribué (administrateur, responsable propreté ou agent/chauffeur).
- Ajouter un utilisateur : création d'un nouveau compte avec définition du rôle, de l'email et du mot de passe initial.
- Modifier un utilisateur : mise à jour des informations personnelles et du rôle d'un compte existant.
- Supprimer un utilisateur : suppression définitive d'un compte du système, avec confirmation préalable.
- Gestion des rôles : attribution des permissions selon trois profils distincts, chacun disposant d'un périmètre d'accès spécifique :
 - *Admin* : accès total à toutes les fonctionnalités.
 - *Responsable propreté* : gestion des tournées, des points et du suivi opérationnel.
 - *Chauffeur / Agent* : accès uniquement à sa tournée assignée via l'application mobile.

Carte des points de collecte (Admin)



Figure 5.12 : Carte interactive des points de collecte — Vue Administrateur

L'interface cartographique constitue l'un des éléments les plus stratégiques de l'application. Elle offre à l'administrateur et au responsable propreté une visualisation géographique en temps réel de l'ensemble des points de collecte répartis sur la zone couverte (ville ou campus).

Fonctionnalités de la carte :

- **Affichage des marqueurs GPS** : chaque point de collecte est représenté sur la carte par un marqueur coloré dont la couleur indique son état :
 - **Vert** : point collecté avec succès.
 - **Orange** : point non encore collecté.
 - **Rouge** : incident signalé sur ce point.
- **Informations au clic** : un clic sur un marqueur affiche une fenêtre contextuelle (*popup*) contenant le nom du point, sa zone, son type (conteneur, bac, corbeille) et le dernier statut enregistré.
- **Filtrage par zone ou tournée** : l'administrateur peut filtrer les points affichés selon la zone géographique ou selon la tournée à laquelle ils sont rattachés.
- **Visualisation des tournées** : les points appartenant à une même tournée peuvent être reliés par un tracé de route afin de matérialiser l'itinéraire prévu pour le chauffeur.
- **Suivi de position du chauffeur** : lorsque la fonctionnalité de géolocalisation est activée sur l'application mobile, la position en temps réel du chauffeur est affichée sur la carte, permettant un suivi dynamique de la tournée.
- **Ajout rapide de points** : l'administrateur peut cliquer directement sur la carte pour créer un nouveau point de collecte à l'emplacement souhaité, sans avoir à saisir manuellement les coordonnées GPS.

Technologies utilisées : La carte est implémentée à l'aide de la bibliothèque Leaflet.js associée aux tuiles de OpenStreetMap, permettant une cartographie interactive, open source et sans frais de licence. Les coordonnées GPS (latitude, longitude) stockées dans la table `collection_points` sont utilisées directement pour positionner chaque marqueur sur la carte.

Cette fonctionnalité renforce considérablement la lisibilité opérationnelle du système : plutôt que de consulter une liste textuelle de points, le responsable dispose d'une vue d'ensemble immédiate et intuitive de l'état de la collecte sur le terrain.

5.4.3 Application mobile (React Native)

Écran de connexion mobile

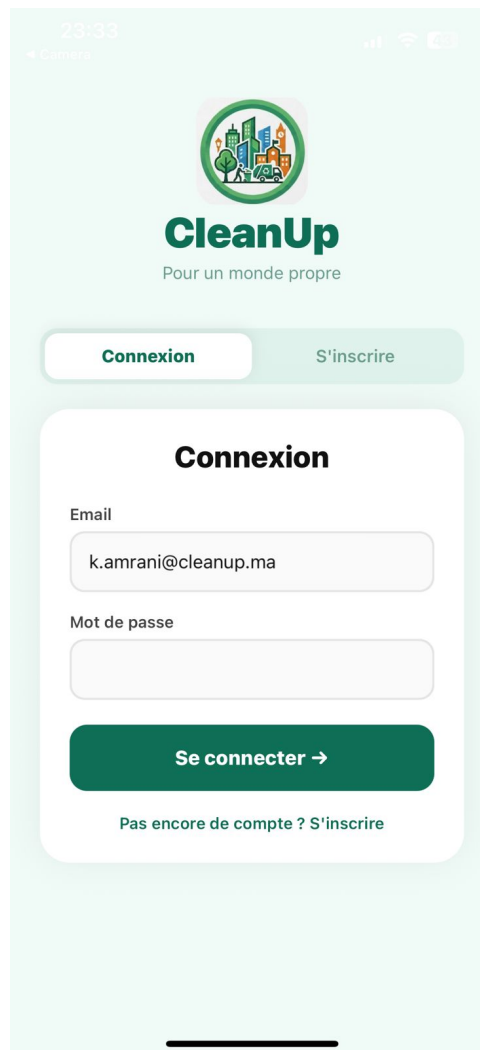


Figure 5.13 : Écran de connexion mobile

Cette interface permet aux agents de se connecter à l'application mobile. Une fois authentifié, l'agent accède à sa tournée quotidienne et aux points de collecte qui lui sont attribués.

L'écran de connexion mobile est optimisé pour une utilisation terrain. Il propose une interface simplifiée avec clavier adaptatif et retour haptique. Le JWT reçu est stocké de façon sécurisée via expo-secure-store.

- Formulaire email/mot de passe avec validation native
- Option 'Rester connecté' via stockage sécurisé du token
- Gestion des erreurs réseau (mode hors ligne)
- Biométrie optionnelle (empreinte digitale)

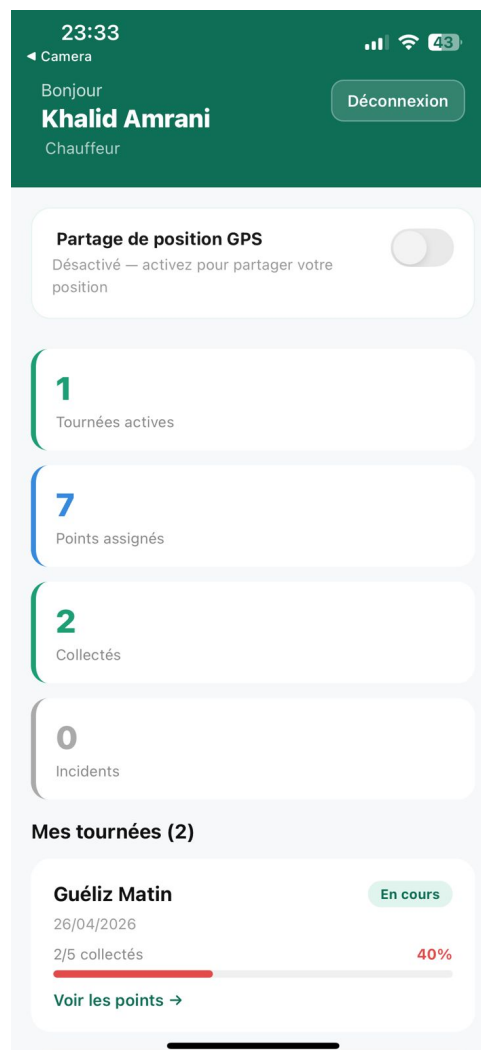
Tableau de bord agent

Figure 5.14 : Tableau de bord agent

Cette interface affiche un résumé des tâches de l'agent :

- Points à collecter
- Points déjà traités
- Statut de la tournée

Le dashboard agent offre une vue synthétique des tâches de la journée, adaptée à une consultation rapide sur le terrain.

Ma tournée (My Route)

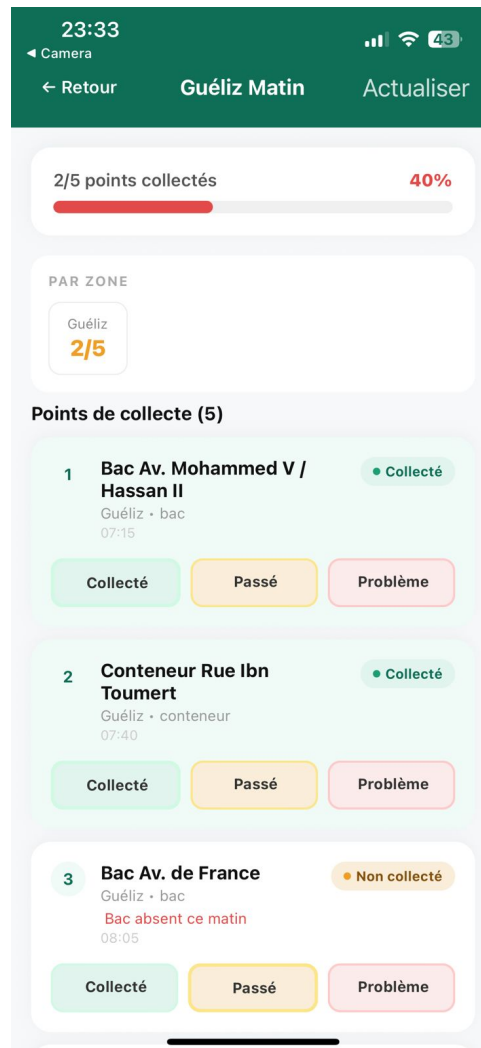


Figure 5.15 : Ma tournée (My Route)

Cette interface permet le suivi de la tournée :

- Liste des points à visiter
- Marquer un point comme collecté / non collecté / problème
- Ajouter une note

Carte et suivi GPS

Cette interface permet la visualisation des points sur une carte géographique et offre :

- Suivi GPS en temps réel avec mise à jour continue de la position
- Visualisation des zones de collecte
- Marqueurs colorés par statut : vert (collecté), orange (en attente), rouge (problème)
- Optimisation du parcours
- Navigation vers le point le plus proche
- Mode économie de batterie pour les longues tournées

Gestion hors ligne

Pour les zones à faible couverture réseau, l'application intègre un mécanisme de synchronisation différée :

1. Les actions de l'agent sont stockées localement dans AsyncStorage
2. Un indicateur visuel signale le mode hors ligne
3. Dès que la connexion est rétablie, les logs en attente sont synchronisés
4. Un message de confirmation informe l'agent de la réussite de la synchronisation

5.5 Stratégie de tests

La qualité du système est garantie par une stratégie de tests multi-niveaux couvrant l'ensemble des composantes de l'application.

5.5.1 Tests d'intégration

Les tests d'intégration valident la communication entre les différents modules du système :

- Tests des endpoints API avec Postman (collections automatisées)
- Vérification des flux complets : authentification → tournée → log → dashboard
- Tests de la cohérence des données entre les tables MySQL
- Vérification des contraintes d'intégrité référentielle

5.5.2 Tests fonctionnels

Les scénarios de test reproduisent les cas d'usage réels définis dans le cahier des charges :

Scénario	Acteur	Résultat attendu	Statut
Connexion avec mauvais mot de passe	Agent	Message d'erreur clair, pas de JWT	Succès (HTT
Créer une tournée et assigner un camion	Responsable	Tournée visible dans le dashboard	Succès (HTT
Marquer un point comme collecté	Agent	Log créé, point vert sur la carte	Succès (HTT
Signaler un problème avec note	Agent	Alerte envoyée au responsable	Succès
Tenter de recollecter sans justification	Agent	Refus de l'API, message d'avertissement	Succès (HTT
Consulter le dashboard	Responsable	KPIs mis à jour en temps réel	Succès (HTT
Exporter l'historique en CSV	Admin	Fichier CSV téléchargé avec toutes les données	SuccèsS

Table 5.6 : Scénarios de tests fonctionnels

5.5.3 Tests de performance

- Temps de réponse de l'API inférieur pour les requêtes courantes
- Chargement du dashboard en moins de 2 secondes
- Application mobile fluide sur Android 10+
- Test de charge : simulation des agents simultanés sur l'API

5.5.4 Tests de sécurité

- Tentatives d'injection SQL sur tous les paramètres GET/POST
- Accès aux endpoints sans JWT → rejet 401
- Accès aux endpoints avec rôle insuffisant → rejet 403
- Test CSRF sur les formulaires web
- Vérification de l'expiration des tokens JWT

5.6 Déploiement et configuration

5.6.1 Configuration locale (développement)

L'environnement de développement local est basé sur XAMPP, permettant de faire tourner Apache et MySQL sans configuration serveur complexe.

Variable	Valeur exemple	Description
DB_HOST	localhost	Hôte MySQL
DB_NAME	CleanUp_db	Nom de la base de données
DB_USER	root	Utilisateur MySQL
JWT_SECRET	***	Clé secrète pour les tokens JWT
JWT_EXPIRY	86400	Durée de vie du token (24h en secondes)
CORS_ORIGIN	http ://localhost :3000	Origines autorisées

Table 5.7 : Variables d'environnement de configuration

5.6.2 Instructions de déploiement

1. Installer XAMPP et démarrer Apache + MySQL
2. Importer le fichier database/schema.sql dans phpMyAdmin
3. Copier le dossier backend/ dans htdocs/ et configurer .env
4. Exécuter npm install && npm start dans frontend/ pour le web
5. Exécuter npx expo start dans mobile/ pour l'app Android
6. Scanner le QR code avec Expo Go pour tester sur mobile réel

5.7 Conclusion

Ce chapitre a présenté les technologies utilisées ainsi que l'architecture globale de l'application de gestion de la collecte des déchets. Depuis le choix du stack technologique (PHP, React, React Native, MySQL) jusqu'à l'implémentation concrète de chaque interface, en passant par la définition d'une architecture client-serveur sécurisée et une stratégie de tests rigoureuse, chaque décision technique a été justifiée par les contraintes fonctionnelles et non-fonctionnelles du cahier des charges.

Le système développé répond intégralement aux objectifs fixés : suivi des points de collecte, gestion des tournées de camions, traçabilité complète via les logs, et visualisation cartographique en temps réel. L'architecture modulaire adoptée facilite les évolutions futures, notamment l'ajout d'algorithmes d'optimisation de tournées, l'intégration de capteurs IoT sur les bacs, ou le passage à une infrastructure cloud.

Points forts du système : interface mobile ergonomique pour les agents, dashboard analytique pour les responsables, API REST sécurisée et documentée, synchronisation hors ligne, et base de données normalisée garantissant l'intégrité des données.

Conclusion générale

La conclusion générale de ce projet met en lumière les principaux enseignements et aboutissements de l'application de collecte des déchets développée dans le cadre de ce travail de fin de modules. Elle permet de récapituler les objectifs initiaux, de synthétiser les contributions réalisées, d'identifier les limites rencontrées et de proposer des perspectives d'amélioration.

Au début du projet, l'objectif principal était de concevoir une solution numérique permettant d'optimiser la gestion de la collecte des déchets dans un environnement urbain ou universitaire. Ce système devait permettre une meilleure organisation des tournées, un suivi en temps réel des opérations, ainsi qu'une centralisation des informations entre les différents acteurs (administrateur, responsables et agents de collecte). L'évaluation du travail réalisé montre que les objectifs fonctionnels et techniques ont été globalement atteints grâce à la mise en place d'une application web et mobile connectée à une base de données centralisée.

Les principales réalisations de ce projet incluent le développement d'une application web pour la gestion des points de collecte, des camions et des tournées, la mise en place d'une application mobile destinée aux agents pour le suivi des opérations sur le terrain, l'intégration d'une base de données MySQL permettant la centralisation et la gestion des données, la création d'un système de suivi des collectes et des incidents en temps réel, ainsi que la mise en place d'un tableau de bord pour la visualisation des statistiques et l'aide à la prise de décision. Ces réalisations ont permis d'améliorer la coordination entre les différents acteurs et de proposer une solution plus moderne et efficace pour la gestion des déchets.

Malgré ces résultats, certaines limites peuvent être relevées, notamment l'absence d'algorithmes avancés d'optimisation automatique des tournées, la dépendance à une connexion Internet pour le fonctionnement en temps réel, des fonctionnalités de suivi encore perfectibles comme la géolocalisation avancée, ainsi qu'un manque d'automatisation intelligente dans la gestion prédictive des déchets. Ces contraintes peuvent influencer l'évolution future du système et constituent des points d'amélioration importants.

Plusieurs perspectives d'évolution peuvent être envisagées pour améliorer ce projet, telles que l'intégration de la géolocalisation en temps réel des camions de collecte, l'utilisation d'algorithmes d'intelligence artificielle pour optimiser les tournées, l'ajout d'un système de notification intelligent pour les incidents et alertes, le développement d'un module citoyen pour signaler les déchets en temps réel, ainsi que la migration vers une architecture plus avancée de type microservices pour améliorer la scalabilité et la maintenance du système.

En conclusion, ce projet représente une expérience enrichissante qui a permis de mettre en pratique les connaissances acquises durant la formation, notamment en développement web, mobile et gestion de bases de données. Il constitue une solution fonctionnelle et évolutive pour la digitalisation de la gestion des déchets, tout en ouvrant la voie à de nombreuses améliorations futures visant à optimiser davantage la propreté urbaine et la

gestion des ressources.

Apport du projet

Au début de ce projet de fin d'études, les objectifs étaient clairement définis vers la conception et le développement de CleanUp, une application de gestion de la collecte des déchets permettant d'optimiser les tournées, d'assurer un suivi en temps réel et d'améliorer la coordination entre agents, responsables et administrateurs. La démarche progressive adoptée, organisée en neuf phases successives selon la planification établie, a permis de structurer le développement de manière itérative et cohérente, depuis l'analyse des besoins jusqu'à la validation finale du système.

Parmi les compétences développées, on peut citer la maîtrise du backend PHP/MVC avec API REST sécurisée par JWT, la conception d'une base de données MySQL relationnelle normalisée, le développement de l'interface web React avec composants réutilisables et tableau de bord temps réel, ainsi que l'application mobile cross-platform React Native / Expo avec géolocalisation GPS intégrée.

Ce projet a présenté des défis spécifiques, notamment l'intégration harmonieuse entre les plateformes web et mobile via une API commune, la gestion des positions GPS en temps réel et l'équilibre entre simplicité terrain et richesse fonctionnelle.

Si le projet était à refaire, des améliorations pourraient être envisagées : l'intégration d'algorithmes d'optimisation des tournées (VRP), un système de notifications push, une architecture microservices et des modules d'intelligence artificielle pour la prédiction des besoins de collecte.

En conclusion, ce projet constitue une expérience enrichissante sur les plans technique et organisationnel, permettant de mettre en pratique les connaissances acquises en développement web et mobile, en bases de données et en modélisation UML, tout en proposant une solution concrète à une problématique réelle de gestion des services de propreté.

Bibliographie

- [1] R. T. Fielding.
« Architectural Styles and the Design of Network-based Software Architectures », *Doctoral dissertation, University of California, Irvine*, 2000.
https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- [2] M. B. Jones, J. Bradley, N. Sakimura.
« JSON Web Token (JWT) », *RFC 7519, Internet Engineering Task Force (IETF)*, 2015.
<https://datatracker.ietf.org/doc/html/rfc7519>
- [3] A. van Kesteren.
« Cross-Origin Resource Sharing (CORS) », *W3C Recommendation*, 2014.
<https://www.w3.org/TR/cors/>
- [4] Niels Provos, David Mazières.
« A Future-Adaptable Password Scheme », *USENIX Annual Technical Conference*, 1999.
<https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>
- [5] Meta Open Source.
« React Native — Learn once, write anywhere ». *Documentation officielle React Native*, 2024.
<https://reactnative.dev/>
- [6] Expo.
« Expo — An open-source platform for making universal native apps ». *Documentation officielle Expo*, 2024.
<https://expo.dev/>
- [7] Expo.
« Expo Router — File-based routing for React Native ». *Documentation Expo Router*, 2024.
<https://expo.github.io/router/docs/>
- [8] Expo.
« Location — Expo SDK Documentation ». *Expo Documentation*, 2024.
<https://docs.expo.dev/versions/latest/sdk/location/>
- [9] Meta Open Source.
« React — A JavaScript library for building user interfaces ». *Documentation officielle React*, 2024.
<https://react.dev/>

- [10] Remix Software.
« React Router — Declarative Routing for React ».
Documentation officielle React Router v6, 2024.
<https://reactrouter.com/en/main>
- [11] Matt Zabriskie.
« Axios — Promise based HTTP client for the browser and node.js ».
Documentation officielle Axios, 2024.
<https://axios-http.com/docs/intro>
- [12] Apprendre le HTML : guides et didacticiels.
MDN Web Docs.
https://developer.mozilla.org/fr/docs/Learn/HTML/Introduction_to_HTML
- [13] Qu'est-ce que le CSS ? Définition et application.
IONOS Digital Guide.
<https://www.ionos.fr/digitalguide/sites-internet/web-design/quest-ce-que-le-css/>
- [14] JavaScript : qu'est-ce que c'est ?
Futura Sciences.
<https://www.futura-sciences.com/tech/definitions/internet-javascript-509/>
- [15] PHP Group.
« PHP : Manuel de référence ».
Documentation officielle PHP, 2024.
<https://www.php.net/manual/fr/>
- [16] PHP Group.
« PHP Data Objects (PDO) — Extension d'accès aux bases de données ».
Documentation officielle PHP, 2024.
<https://www.php.net/manual/fr/book.pdo.php>
- [17] MySQL, tout comprendre au logiciel de gestion de données relationnelles.
DataScientest.
<https://datascientest.com/mysql-tout-comprendre>
- [18] Oracle Corporation.
« MySQL 8.0 Reference Manual ».
Documentation officielle MySQL, 2024.
<https://dev.mysql.com/doc/refman/8.0/en/>
- [19] B. Hofmann-Wellenhof, H. Lichtenegger, E. Wasle.
GNSS – Global Navigation Satellite Systems : GPS, GLONASS, Galileo, and more.
Springer Vienna, 2008.
- [20] AMCS Group.
« AMCS Platform — Waste Management Software ».
<https://www.amcsgroup.com/solutions/waste-management-software/>
- [21] Routemaster.
« Routemaster — Route Planning and Fleet Management Software ».
<https://www.routemaster.app/>
- [22] Trackunit.
« Trackunit — Fleet Management and IoT Telematics ».
<https://trackunit.com/>

- [23] Praxedo.
« Praxedo — Field Service Management Software ».
<https://www.praxedo.com/fr/>
- [24] Webnet.
« Visual Studio Code - Le blog technique Webnet ».
<https://blog.webnet.fr/visual-studio-code/>
- [25] Apache Friends.
« About the XAMPP project ».
<https://www.apachefriends.org/fr/about.html>
- [26] Postman, Inc.
« Postman API Platform ».
<https://www.postman.com/>
- [27] Git SCM.
« Git — Documentation officielle ».
<https://git-scm.com/doc>
- [28] UML : un langage de modélisation de type graphique.
IONOS Digital Guide.
<https://www.ionos.fr/digitalguide/sites-internet/developpement-web/uml-un-langage-de-modelisation-pour-la-programmation-orientee-objet/>



Sahmad Fatima-ezzahra

Résumé du Projet

Le présent projet a pour objectif de concevoir et développer une application dédiée à la gestion de la collecte des déchets, visant à améliorer l'organisation des tournées, le suivi des points de collecte et l'efficacité des interventions sur le terrain.

Les principales contributions de ce projet résident dans la mise en place d'une interface utilisateur simple et intuitive destinée aux agents (application mobile React Native) et aux responsables (dashboard web React), ainsi que dans le développement d'un système fiable de gestion des données (API REST PHP/ MySQL) permettant de suivre en temps réel l'état des points de collecte, des camions et des itinéraires. La solution intègre nativement la géolocalisation des points et le suivi GPS des agents.

La solution a été validée à travers plusieurs tests (fonctionnels, performance, sécurité), démontrant une amélioration notable en termes d'organisation, de réactivité et d'optimisation des ressources par rapport aux méthodes traditionnelles.

Mots clés : collecte des déchets, gestion des tournées, points de collecte, application web et mobile, suivi en temps réel, gestion des données.